

Rapport de stage

Benchmarking interactif, développement de scRAW et évaluation inductive pour le clustering scRNA-seq

Fabien Bidet

Avril 2026

Période de stage : Janvier-Juin 2026

Organisme d'accueil : LaBRI (Laboratoire Bordelais de Recherche en Informatique)

Responsables de stage :

- **Adélaïde Raguin** – Chargée de recherche CNRS, chercheuse au LaBRI (Tutrice de stage)
- **Victoria Bourgeois** – Maître de Conférences, chercheuse au LaBRI, Université de Bordeaux (Encadrante universitaire)
- **Loann Giovannangeli** – Maître de Conférences, chercheur au LaBRI, Université de Bordeaux (Encadrant universitaire)
- **Patricia Thébault** – Professeure des Universités, chercheuse au LaBRI, Université de Bordeaux (Encadrante universitaire)

Table des matières

1	Introduction	3
1.1	Environnement de travail	3
1.2	Introduction du sujet de stage	4
2	État de l’art	5
2.1	Données	5
2.2	Traitements de données	7
2.2.1	Pré-process (Le nettoyage initial)	7
2.2.2	Batch effect (L’effet de lot)	7
2.3	Algorithmes de clustering	8
2.3.1	Approches classiques	8
2.3.2	Approches Deep Learning (Apprentissage Profond)	8
2.3.3	Les types cellulaires dans les algorithmes	10
3	Comparaison de l’existant	11
3.1	Logiciel	12
3.2	Expériences	14
3.2.1	Données et splits	14
3.2.2	Méthodes	15
3.2.3	Résultats	17
4	scRAW	18
4.1	Positionnement et motivation	18
4.2	Pipeline	19
4.3	Fonctions de perte et variantes	20
4.4	Recherche d’hyperparamètres	22
4.5	Résultats et discussion	22
5	Travaux complémentaires	25
5.0	Transfert de loss	25
5.1	Induction	26
5.2	Interprétation biologique ?	26
	Conclusion finale à rédiger	26
	Figures et tableaux restants à préparer	26
A	Structure et organigramme du LaBRI	28
B	Détails des datasets de l’état de l’art	30
C	Tableau des métriques d’évaluation	31
D	Résultats complets sur les huit jeux de données communs	32

E	Étude d’ablation de scRAW	39
F	Hyperparamètres de la configuration de référence scRAW	42
G	Figures de recherche d’hyperparamètres scRAW	44
H	Screenshots de l’interface graphique SCRBenchmark	47

1 Introduction

1.1 Environnement de travail

Ce stage de fin d'études s'est déroulé du 4 janvier au 30 juin 2026 au sein du Laboratoire Bordelais de Recherche en Informatique (LaBRI). J'ai intégré l'équipe Bench to Knowledge and Beyond (BKB), dirigée par Romain Bourqui (responsable d'équipe) et Patricia Thébault (adjointe). Cette équipe est rattachée au département Systèmes et Données (SeD). Afin de mieux appréhender la structure dans laquelle j'ai évolué, l'organisation détaillée du laboratoire ainsi que son organigramme structurel sont présentés en Annexe A (Figure 5).

Équipe d'encadrement

Au cours de ce stage, j'ai été encadré par une équipe aux expertises complémentaires :

- **Victoria Bourgeais**, enseignante-chercheuse au LaBRI, experte dans l'application de l'apprentissage profond au domaine de la santé ;
- **Loann Giovannangeli**, enseignant-chercheur au LaBRI, en visualisation de l'information et intelligence artificielle ;
- **Patricia Thébault**, enseignante-chercheuse au LaBRI, en sciences des données omiques et réseaux biologiques.

Cadre de travail

J'ai effectué mes travaux dans un bureau partagé avec des doctorants et d'autres stagiaires. Ce contexte s'est révélé particulièrement enrichissant, facilitant les échanges de connaissances au quotidien et me permettant de mieux cerner les enjeux d'une thèse et de la vie de jeune chercheur(se).

Par ailleurs, j'ai pu assister aux séminaires hebdomadaires organisés par les doctorant(e)s de l'équipe BKB, durant lesquels des chercheurs invités présentaient leurs travaux en bioinformatique. J'ai également eu l'opportunité de participer aux séminaires d'autres équipes, notamment celles spécialisées en intelligence artificielle. Ces événements ont constitué une source d'apprentissage scientifique de pointe, idéale pour mon profil situé à l'interface entre la bioinformatique et l'intelligence artificielle. D'un point de vue technique, j'ai bénéficié d'un accès distant (via SSH) à un serveur de calcul interne au LaBRI pour réaliser mes expérimentations.

Suivi et méthodologie

L'avancement de mes travaux a été rythmé par des réunions régulières. Plusieurs réunions mensuelles étaient organisées avec mes encadrants pour faire le point sur l'avancement scientifique. De plus, des réunions bimensuelles ont été planifiées avec Elodie Darbot et Cyril Dourthe, deux bioinformaticien(ne)s et chercheur en bioinformatique, et plusieurs stagiaires, afin de discuter et de présenter nos avancements dans nos projets. Enfin, j'ai pu compter sur la disponibilité de ma tutrice de stage Adélaïde Raguin, chercheuse au LaBRI, pour faire le point sur mon avancement et répondre à mes interrogations tout au long de cette période.

1.2 Introduction du sujet de stage

Le séquençage de l'ARN à l'échelle de la cellule unique (*single-cell RNA sequencing*, scRNA-seq) a profondément transformé l'analyse des tissus biologiques. Contrairement aux approches transcriptomiques classiques, comme l'analyse bulk-ARN, qui mesurent une expression moyenne sur un ensemble de cellules, le scRNA-seq permet d'observer les profils d'expression cellule par cellule. Il devient alors possible d'étudier l'hétérogénéité interne d'un tissu, d'identifier différents types ou états cellulaires, et de mettre en évidence des populations minoritaires qui auraient été masquées dans une mesure globale [1, 2, 3, 4, 5].

Cette richesse d'information s'accompagne cependant de difficultés importantes. Les données scRNA-seq sont de très grande dimension, car chaque cellule est décrite par l'expression de plusieurs milliers de gènes. Elles sont également bruitées, très creuses, sensibles aux événements de non-détection (effet dropout) et souvent affectées par des effets de batch liés au protocole expérimental, au laboratoire ou à la plateforme de séquençage [6]. Avant toute interprétation biologique, il est donc nécessaire de construire une représentation plus compacte et plus robuste des cellules.

Dans ce contexte, le clustering constitue une étape centrale des analyses scRNA-seq. Il consiste à regrouper les cellules présentant des profils d'expression similaires afin de retrouver des types cellulaires, des sous-types ou des états fonctionnels. Des pipelines classiques, comme ceux fondés sur une réduction de dimension suivie d'un clustering, sont aujourd'hui largement utilisés dans les outils bioinformatiques standards [7, 8]. Plus récemment, les méthodes d'apprentissage profond ont cherché à apprendre des espaces latents plus adaptés à la complexité des données, notamment à l'aide d'autoencodeurs ou de modèles probabilistes [9].

Un des enjeux abordés pendant ce stage concerne plus spécifiquement les populations cellulaires rares. En biologie, ces populations peuvent correspondre à des cellules immunitaires minoritaires, à un sous-type tumoral, à une population pathologique, à un état transitoire de différenciation ou encore à des cellules spécifiques d'un tissu. Leur faible proportion ne signifie donc pas qu'elles sont secondaires : au contraire, elles peuvent porter un signal biologique essentiel. Pourtant, du point de vue algorithmique, elles sont particulièrement fragiles. Comme elles représentent peu d'observations, elles contribuent faiblement à l'apprentissage des modèles et peuvent facilement être absorbées et confondues par une population majoritaire ou par un type cellulaire proche. Un bon score moyen dans une métrique généraliste peut alors masquer un échec important : la disparition d'une population biologiquement pertinente.

L'enjeu de ce mémoire est alors de développer une solution unique qui permettrait à la fois d'obtenir un clustering global précis, tout en permettant l'identification des cellules rares et ce même face à des jeux de données comportant un bruit technique important et de pouvoir utiliser ce modèle sur de nouvelles données non vues pendant l'entraînement. Ce dernier point est particulièrement important : dans un contexte réel, il peut être pertinent d'utiliser un modèle déjà entraîné car cette tâche peut être très coûteuse si il faut la répéter à chaque fois que l'on veut utiliser notre modèle. Il doit pouvoir projeter ou classer de nouvelles cellules, issues par exemple d'un nouveau patient, d'un nouveau batch ou d'une expérience complémentaire. On distinguera alors les approches transductives, qui travaillent uniquement

avec les données disponibles, et inductives, qui peuvent généraliser à de nouvelles données.

Mon stage s'est inscrit dans cette problématique à l'interface entre bioinformatique, apprentissage automatique et évaluation expérimentale. Le travail réalisé s'est articulé autour de quatre contributions principales. La première a consisté à développer et structurer **SCRBenchmark**, une interface web interactive destinée à comparer différentes méthodes de clustering scRNA-seq sur plusieurs jeux de données. La deuxième a porté sur la comparaison de méthodes existantes, classiques et fondées sur l'apprentissage profond, en intégrant des métriques globales mais aussi des métriques centrées sur les classes rares. La troisième contribution a été le développement de **scRAW**, une méthode visant à apprendre une représentation latente en donnant davantage d'importance aux cellules rares ou peu denses pendant l'apprentissage. Enfin, des travaux complémentaires ont exploré le transfert de cette logique de pondération vers d'autres pertes et les premiers tests d'un comportement inductif.

La question directrice du mémoire peut ainsi se formuler de la manière suivante : comment construire, évaluer et améliorer un pipeline de clustering scRNA-seq capable de mieux préserver les populations cellulaires rares, tout en restant reproductible et utilisable sur de nouvelles données ? Pour y répondre, la suite du rapport présente d'abord les notions nécessaires à la compréhension des données scRNA-seq et des méthodes de clustering. Elle décrit ensuite le protocole de comparaison mis en place, puis détaille la conception et l'évaluation de scRAW. Enfin, elle discute les extensions explorées pendant le stage, notamment le transfert de loss, l'induction et les pistes d'interprétation biologique.

2 État de l'art

L'objectif de cette section est de présenter les bases nécessaires à la compréhension du rapport pour mieux comprendre mes contributions et où elles se situent par rapport à la recherche actuelle. Nous commencerons par examiner les jeux de données utilisés, puis nous verrons comment ces données sont préparées. Enfin, nous ferons une présentation des algorithmes de clustering existants, en comprenant notamment pourquoi les modèles d'intelligence artificielle actuels ont souvent du mal à repérer les cellules rares.

2.1 Données

Pour qu'une méthode de clustering soit considérée comme robuste, elle doit faire ses preuves sur des données variées. Dans le cadre de ce stage, nous nous sommes appuyés sur la collection de huit jeux de données du sous-ensemble commun (détaillés en Annexe B).

Ces données proviennent de différents organes et espèces : pancréas humain, cerveau, rate, sang (PBMC), testicules, foie, moelle osseuse, ou encore rétine de macaque. Chaque jeu de données possède ses propres défis :

- **La taille** : Cela va de petits jeux de données d'environ 2 700 à 3 000 cellules (comme *Paul15 bone marrow* ou *Tabula Muris liver*) jusqu'à de très gros ensembles de cellules dépassant les 30 000 cellules (comme la rétine de macaque ou les PBMC).
- **Les effets de batch** : La plupart des jeux de données sont composés de plusieurs "batches" (lots expérimentaux). Par exemple, les données de PBMC combinent 16 lots

différents, et la rétine de macaque en compte 30.

- **Les populations rares** : C'est le cœur de notre problématique. Presque tous nos jeux de données contiennent des types cellulaires dits "rares" (représentant moins de 5 % des cellules) ou "ultra-rares" (moins de 1 %). Par exemple, dans les données du pancréas humain de Baron, certaines cellules immunitaires ou cellules de Schwann ne représentent qu'une fraction de pourcent du total.

L'utilisation d'annotations pré-existantes (la "vérité terrain") nous permet d'évaluer objectivement si les algorithmes parviennent à retrouver ces populations ou s'ils les noient dans la masse. Le tableau 5 (disponible en Annexe B) résume l'ensemble des huit jeux de données de référence retenus pour notre étude, détaillant pour chacun d'eux l'espèce d'origine, le nombre de cellules et de gènes, le nombre de lots (batchs), la quantité de populations cellulaires rares, ainsi que leur effet de lot (PCR sur PCA) et déséquilibre de classe (indice de Gini).

Les données de séquençage d'ARN sur cellule unique (scRNA-seq) manipulées sont généralement structurées sous forme de matrices d'expression accompagnées de métadonnées. Par convention (notamment dans le format `AnnData` utilisé par notre pipeline), ces informations sont organisées de la façon suivante :

- **La matrice d'expression (X)** : De dimension $N \times G$ (où N est le nombre de cellules et G le nombre de gènes). Chaque entrée représente le niveau d'expression (par exemple le nombre de comptages d'U.M.I., pour *Unique Molecular Identifiers*) d'un gène spécifique dans une cellule donnée. Cette matrice est généralement très creuse (majoritairement remplie de zéros).
- **Les métadonnées cellulaires (*observations* ou *obs*)** : Une table de dimension $N \times C$ alignée sur les lignes de la matrice d'expression. Elle attribue à chaque cellule des informations de contexte (comme le lot expérimental d'origine ou l'annotation biologique) réparties sur C colonnes.
- **Les métadonnées des gènes (*variables* ou *var*)** : Une table de dimension $G \times V$ alignée sur les colonnes de la matrice d'expression, décrivant les caractéristiques des gènes (nom du gène, niveau de variabilité globale, etc.) réparties sur V colonnes.

Le tableau 1 illustre un exemple conceptuel de cet alignement entre la matrice d'expression et les métadonnées cellulaires.

Identifiant Cellule	Matrice d'expression (X)			Métadonnées des cellules (<i>obs</i>)	
	Gène A	Gène B	Gène C	Lot (Batch)	Type cellulaire
Cellule_1	0	14	0	Lot_A	Cellule Beta
Cellule_2	3	0	1	Lot_A	Cellule Alpha
Cellule_3	0	0	0	Lot_B	Cellule de Schwann
Cellule_4	24	1	0	Lot_B	Cellule Beta
Cellule_5	0	0	8	Lot_C	Lymphocyte T

TABLE 1 – Structure conceptuelle d'un jeu de données scRNA-seq. La matrice d'expression à gauche est alignée ligne à ligne (par cellule) avec la table des métadonnées cellulaires à droite.

2.2 Traitements de données

Les données brutes issues du séquençage scRNA-seq sont extrêmement bruitées et pleines de zéros (des gènes non détectés). Il est donc indispensable de les nettoyer et de les préparer.

2.2.1 Pré-process (Le nettoyage initial)

Le traitement standard [6] consiste d'abord à filtrer les cellules de mauvaises qualités et les gènes très peu exprimés. Ensuite, on normalise les données pour que toutes les cellules soient comparables, puis on applique une transformation mathématique (le logarithme) pour atténuer les écarts extrêmes. Enfin, on ne conserve généralement que les gènes dits "hautement variables" (HVG), c'est-à-dire ceux qui différencient le mieux les cellules entre elles. Bien que ces étapes soient standardisées, elles présentent un risque pour les cellules rares : un filtrage trop agressif peut effacer les quelques gènes qui caractérisaient spécifiquement une toute petite population.

2.2.2 Batch effect (L'effet de lot)

L'effet de batch est un biais technique. Si l'on séquence le même tissu dans deux laboratoires différents, ou sur deux machines différentes, les cellules auront tendance à se regrouper par "laboratoire" plutôt que par "type cellulaire". Il existe des méthodes dédiées pour corriger ce problème (présentées dans le tableau 2), comme ComBat [10], Harmony [11], Scanorama [12], ou scVI [9]. L'objectif est d'aligner les différents lots pour effacer la signature technique. Cependant, la difficulté est de ne pas "sur-corriger" : en voulant effacer les différences techniques, on risque de gommer de vraies différences biologiques, en particulier celles des populations rares qui pourraient n'apparaître que dans un seul lot.

Une stratégie plus récente, utilisée ensuite dans notre modèle, consiste à apprendre une représentation qui conserve l'information utile tout en rendant le batch difficile à reconnaître. C'est l'idée des DANN (*Domain-Adversarial Neural Networks*) : un réseau principal apprend l'espace latent, tandis qu'un second réseau essaie de prédire le domaine ou le lot expérimental à partir de cet espace. Grâce à une couche d'inversion de gradient, l'encodeur est pénalisé lorsque le batch reste trop facile à prédire, ce qui l'encourage à produire une représentation moins dépendante du lot [13]. Cette logique a déjà été appliquée à des données biologiques, notamment avec scDGN pour aligner des données scRNA-seq entre expériences [14], puis avec D2AN pour associer correction de batch et annotation des types cellulaires [15]. Des approches proches, comme ABC, ajoutent également un discriminateur de batch à un autoencodeur afin de limiter l'information technique dans l'espace latent [16]. Le point de vigilance reste le même que pour les autres méthodes : si le batch est fortement corrélé à une différence biologique réelle, la correction adversariale peut aussi affaiblir ce signal.

Outil	Méthodologie	Principe de correction
ComBat [10]	Modèle bayésien empirique	Ajustement des moyennes et variances des gènes par lot
DANN [13]	Réseau de neurones adversarial	Inversion de gradient pour masquer l'information de lot dans l'espace latent
Harmony [11]	Soft K-Means itératif	Alignement des cellules similaires de lots différents en PCA
Scanorama [12]	Mutual Nearest Neighbors (MNN)	Identification des cellules "panoramas" communes entre les lots
scVI [9]	Autoencodeur Variationnel (VAE)	Modélisation probabiliste du bruit et des batchs dans l'espace latent

TABLE 2 – Principaux outils de correction de l'effet de lot (Batch Effect).

2.3 Algorithmes de clustering

Une fois les données prêtes, l'objectif est de regrouper les cellules similaires.

2.3.1 Approches classiques

L'approche la plus courante en bioinformatique consiste à simplifier les données (généralement via une méthode appelée PCA), puis à regrouper directement les cellules avec un algorithme comme K-Means [17], ou bien à construire un graphe où chaque cellule est reliée à ses voisines les plus proches pour y identifier des communautés avec des algorithmes comme Louvain [18] ou Leiden [19]. Ces méthodes sont très rapides, faciles à utiliser et intégrées dans des outils standards comme Scanpy [7]. Toutefois, elles sont très sensibles au nettoyage initial des données. De plus, si une population est très petite, elle risque de ne pas former sa propre communauté dans le graphe et d'être absorbée par une population voisine plus grande.

2.3.2 Approches Deep Learning (Apprentissage Profond)

Pour aller plus loin, les méthodes basées sur le Deep Learning (l'intelligence artificielle) apprennent à représenter les cellules dans un nouvel espace mathématique (l'espace latent) de manière beaucoup plus complexe et performante que la PCA. Ces méthodes, souvent basées sur des autoencodeurs, s'entraînent en essayant de minimiser une "fonction de perte" (ou *loss*). Cette *loss* est une information mathématique qui évalue la qualité de l'apprentissage et pénalise le modèle lorsqu'il commet des erreurs.

Pour bien comprendre, regardons sur quoi se base la perte totale (*total loss*) de différents modèles de l'état de l'art :

- **La perte de reconstruction** : Des modèles comme scVI [9] ou scMAE [20] cherchent avant tout à bien reconstruire le profil d'expression initial de la cellule. Si le modèle

compresse mal l'information et n'arrive pas à deviner les gènes de départ, sa perte augmente.

- **La perte de clustering :** Des méthodes comme DESC [21], scDeepCluster [22] ou scNAME [23] ajoutent une contrainte supplémentaire. Elles mesurent les distances entre les cellules et forcent le modèle à regrouper les cellules qui se ressemblent de plus en plus au fil du temps (souvent en minimisant une mesure de différence appelée divergence KL).

Le cœur du problème avec les populations rares vient précisément de la manière dont cette perte totale est calculée. Le modèle cherche à minimiser cette perte globale sur l'ensemble des données. Les populations majoritaires (qui contiennent des milliers de cellules) vont avoir un poids énorme dans ce calcul. À l'inverse, si le modèle se trompe sur un type cellulaire rare (qui ne contient que quelques dizaines de cellules), cela fera à peine bouger la perte globale. Le réseau de neurones va donc avoir tendance à "ignorer" la précision sur les cellules rares pour optimiser sa représentation mathématique globale.

Pour résumer, le tableau 3 présente une synthèse des méthodes citées, des approches classiques jusqu'aux architectures Deep Learning et aux méthodes dédiées aux populations rares.

Famille	Méthode	Framework	Objectif / loss	Réf.	Source
Approches classiques	PCA	Projection linéaire	Maximisation de la variance expliquée	[24]	Philosophical Magazine
	K-Means	Centroïdes	Minimisation de l'inertie intra-cluster	[17]	Berkeley Symposium
	Louvain	Graphe de voisinage	Maximisation de la modularité	[18]	Journal of Statistical Mechanics
	Leiden	Graphe de voisinage	Modularité avec communautés mieux connectées	[19]	Scientific Reports
Deep Learning	scVI	VAE probabiliste	Reconstruction NB/ZINB + divergence KL	[9]	Nature Methods
	scMAE	Masked autoencoder	Reconstruction de gènes masqués	[20]	Bioinformatics
	DESC	Autoencodeur + clustering	Reconstruction + clustering par divergence KL	[21]	Nature Communications
	scDeepCluster	Autoencodeur ZINB	Reconstruction ZINB + clustering par divergence KL	[22]	Nature Machine Intelligence
	scNAME	Contrastive learning	Reconstruction + clustering + contraste de voisinage	[23]	Bioinformatics
Graphes profonds	scCDCG	AE + graph embedding	Reconstruction + transport optimal + graph cut	[25]	DASFAA
Cellules rares	GiniClust	Indice de Gini	Détection de gènes marqueurs atypiques	[26]	Genome Biology
	CellSIUS	Signatures locales	Identification de sous-populations minoritaires	[27]	Genome Biology
	scAIDE	Autoencodeur	Débruitage + clustering orienté populations rares	[28]	NAR Genomics and Bioinformatics
	DeepScena	Self-supervised	Raffinement des clusters rares	[29]	Briefings in Bioinformatics
	scCAD	Détection d'anomalies	Décomposition de clusters pour isoler les types rares	[30]	Nature Communications

TABLE 3 – Synthèse des méthodes de clustering et de détection de populations rares citées dans l'état de l'art. AE : autoencodeur ; VAE : autoencodeur variationnel ; KL : divergence de Kullback-Leibler ; NB/ZINB : lois binomiale négative et binomiale négative à inflation de zéros.

2.3.3 Les types cellulaires dans les algorithmes

Face à ces limites, l'identification des populations cellulaires minoritaires est devenue un enjeu central dans l'analyse des données single-cell RNA-seq. Plusieurs algorithmes ont ainsi été développés pour mieux détecter ces sous-populations rares. Tous s'appuient, d'une manière ou d'une autre, sur les profils d'expression des gènes, mais ils ne les exploitent pas de la même façon.

Certaines méthodes cherchent directement des gènes caractéristiques des cellules rares. Par exemple, **GiniClust** [26] utilise l'indice de Gini pour repérer des gènes exprimés dans seulement un petit nombre de cellules. Ces gènes peuvent alors signaler l'existence d'une

population rare qui serait difficile à voir avec un clustering classique. Dans une logique proche, **CellSIUS** [27] part de groupes cellulaires déjà formés, puis recherche à l'intérieur de ces groupes des signatures d'expression particulières permettant d'identifier des sous-populations minoritaires.

D'autres méthodes cherchent plutôt à améliorer la représentation globale des cellules avant de les regrouper. C'est le cas de **scAIDE** [28], qui utilise un autoencodeur pour réduire le bruit des données et produire une représentation plus claire des cellules. Une fois cette représentation obtenue, scAIDE applique une méthode de clustering conçue pour limiter le biais en faveur des grandes populations cellulaires. L'objectif est donc de mieux séparer les petits groupes de cellules, y compris lorsqu'ils sont très peu représentés dans le jeu de données.

Dans une approche proche, mais davantage hiérarchique, **DeepScena** [29] commence par identifier de grands groupes cellulaires, puis réanalyse progressivement certains de ces groupes afin de faire apparaître des sous-populations plus fines. À chaque niveau, les gènes les plus informatifs peuvent être recalculés dans le sous-groupe étudié, ce qui permet de mieux détecter des différences qui étaient invisibles à l'échelle globale. DeepScena ne donne donc pas directement plus de poids aux cellules rares dans sa fonction de perte ; il les révèle plutôt grâce à un raffinement progressif du clustering et à une meilleure modélisation des données single-cell.

Enfin, des méthodes plus récentes comme **scCAD** [30] abordent la question sous l'angle de la détection d'anomalies. L'idée est d'identifier les cellules dont le profil d'expression s'éloigne de celui des populations majoritaires, puis de décomposer progressivement les clusters afin d'isoler ces groupes atypiques.

Ces méthodes reposent donc sur une idée commune : un clustering global classique tend à favoriser les grandes populations cellulaires et peut masquer les cellules rares ou les états intermédiaires. Notre démarche, **scRAW**, s'inscrit dans cette continuité tout en proposant une stratégie différente. Plutôt que d'identifier les populations rares uniquement après la construction des clusters ou par une étape de raffinement, nous cherchons à intégrer cette contrainte directement pendant l'apprentissage du modèle. Concrètement, scRAW attribue un poids plus important aux cellules isolées ou faiblement représentées dans le calcul de la *loss*, afin qu'elles influencent davantage la construction de l'espace latent. L'objectif est ainsi de produire une représentation latente plus informative, dans laquelle les populations rares ne sont pas absorbées par les grands groupes cellulaires et étant mise en évidence tout au long de l'entraînement du modèle.

3 Comparaison de l'existant

Nos travaux ont débuté par une comparaison des méthodes de l'état de l'art traitant des données de séquençage d'ARN à cellule unique (scRNA-seq). À ce stade, nous souhaitions étudier des approches généralistes, qui ne traitaient ni de l'effet de lot (*batch effect*) ni du déséquilibre des populations cellulaires. Pour ce faire, nous nous sommes en partie appuyés sur le protocole et les sélections du benchmark scCluBench [31] où nous avons sélectionné les méthodes de deep learning basés sur des autoencodeurs et sur des graphs les plus performantes sur les métriques comparées. La problématique ensuite reconstruite pour comparer les méthodes

sélectionner était la façon dont on voulait effectuer les tests. Nous souhaitions à la fois pouvoir modifier à notre guise les hyperparamètres des méthodes, tout en ajustant les étapes de preprocessing. De plus la quantité des tests différents et des résultats disponibles m'a poussé à développer une solution logiciel le plus ergonomique possible, SCRBenchmark, afin d'automatiser, simplifier la mise en place des différentes expériences et l'exploration des résultats.

3.1 Logiciel

SCRBenchmark a été développé pour rendre les expériences de comparaison plus simples à construire, plus faciles à reproduire et plus lisibles pour des utilisateurs qui ne travaillent pas nécessairement dans le code au quotidien. Le logiciel regroupe dans un même environnement le chargement des données, le choix du protocole expérimental, le prétraitement, la sélection des méthodes, l'exécution des analyses et l'exploration des résultats. Sa structure logique est résumée dans la figure 1.

Le premier choix important a été de séparer l'interface utilisée pour configurer les expériences du moteur chargé d'exécuter les calculs. L'interface web guide l'utilisateur étape par étape, tandis que la ligne de commande permet de relancer la même analyse sur un serveur ou dans un environnement de calcul plus adapté aux expériences longues. Les neuf écrans principaux de l'interface, consultables en Annexe H (Figure 12), couvrent l'ensemble du parcours : importation des données, visualisation préliminaire, choix du type d'expérience, prétraitement, sélection des algorithmes, génération de la commande d'exécution, suivi du calcul, consultation des métriques et visualisation des UMAP finales.

SCRBenchmark propose ensuite deux modes d'analyse. Le mode standard utilise toutes les cellules ensemble et sert à observer directement le comportement d'une méthode sur un jeu de données. Le mode benchmark sépare au contraire les cellules en ensembles d'entraînement, de validation et de test, afin d'évaluer les méthodes dans des conditions plus contrôlées, notamment lorsqu'il s'agit de tester leur comportement sur des cellules ou des lots non vus pendant l'entraînement.

L'intérêt principal du logiciel est d'imposer un cadre commun à des méthodes très différentes. Le même pipeline de prétraitement peut être appliqué à toutes les méthodes : filtrage, normalisation, sélection des gènes les plus informatifs et, si nécessaire, correction de l'effet de lot. De la même manière, le moteur d'exécution attend de chaque algorithme les mêmes types de sorties : groupes de cellules prédits, représentation numérique des cellules lorsque la méthode en produit une, paramètres utilisés et temps d'exécution. Ce processus commun permet de s'assurer que les résultats seront issus du fonctionnement des algorithmes et non des données en entrée.

Enfin, SCRBenchmark sauvegarde les éléments nécessaires à l'analyse et à la reproductibilité : scores, labels prédits, embeddings, figures, paramètres et logs. Les visualisations générées automatiquement, comme les UMAP, les matrices de confusion ou les figures comparant entraînement et test, permettent d'explorer rapidement les résultats sans réécrire du code pour chaque expérience. L'outil a donc servi à la fois à produire les expériences et à en analyser les sorties de manière cohérente.

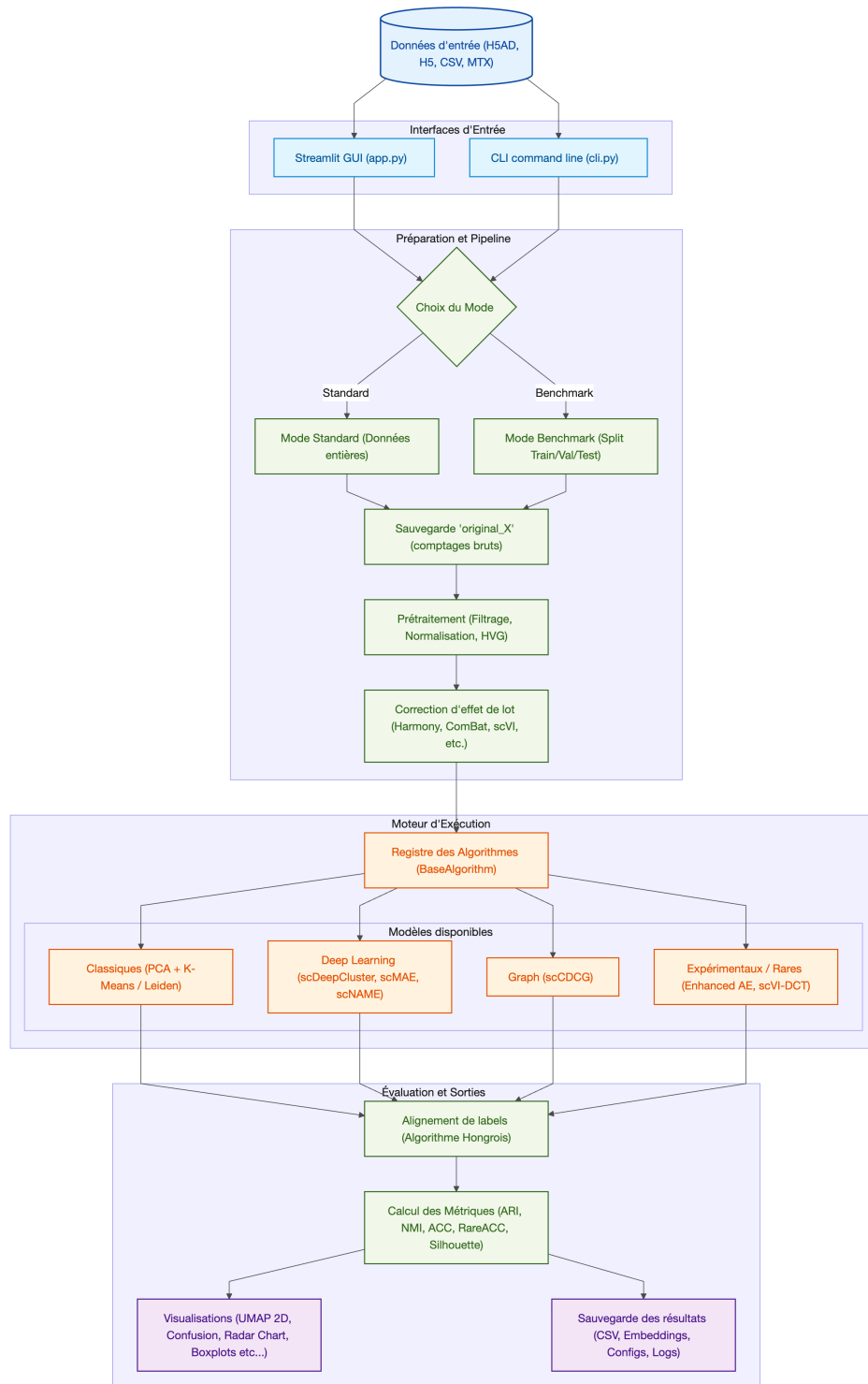


FIGURE 1 – Structure logique et flux de travail de la plateforme de benchmarking SCRBenchmark.

3.2 Expériences

3.2.1 Données et splits

Nous avons retenu le jeu de données Baron [32] pour les expériences présentées ici. Ce choix permettait de réunir dans un même cadre les deux difficultés au centre du mémoire : un effet de lot lié aux donneurs et un fort déséquilibre entre types cellulaires. Le jeu de données regroupe quatre donneurs humains, notés *human1* à *human4*, et constitue un standard de référence largement utilisé pour évaluer les méthodes de clustering en scRNA-seq. Les publications originales de plusieurs méthodes d'apprentissage profond comme scDeepCluster [22], scNAME [23], scMAE [20] et scCDCG [25], ainsi que des approches dédiées aux populations rares telles que CellSIUS [27], scAIDE [28], DeepScena [29], scCAD [30] et le benchmark scCluBench [31], s'appuient sur ce type de données de référence pour valider leurs performances.

Après filtrage, les expériences sur Baron portent sur 8 569 cellules réparties en quatorze types cellulaires. Le tableau 4 montre que les cellules *alpha* et *beta* représentent à elles seules plus de la moitié du jeu de données, tandis que plusieurs populations d'intérêt biologique, comme les *macrophage*, *mast*, *schwann* ou *t_cell*, représentent moins de 1 % des cellules. Ce jeu de données est donc particulièrement adapté pour tester si un algorithme conserve les petites populations au lieu d'optimiser uniquement les grandes classes.

Type cellulaire	Cellules	Proportion	Statut
<i>beta</i>	2 525	29,47 %	Fréquente
<i>alpha</i>	2 326	27,14 %	Fréquente
<i>ductal</i>	1 077	12,57 %	Fréquente
<i>acinar</i>	958	11,18 %	Fréquente
<i>delta</i>	601	7,01 %	Fréquente
<i>activated_stellate</i>	284	3,31 %	Rare
<i>gamma</i>	255	2,98 %	Rare
<i>endothelial</i>	252	2,94 %	Rare
<i>quiescent_stellate</i>	173	2,02 %	Rare
<i>macrophage</i>	55	0,64 %	Ultra-rare
<i>mast</i>	25	0,29 %	Ultra-rare
<i>epsilon</i>	18	0,21 %	Ultra-rare
<i>schwann</i>	13	0,15 %	Ultra-rare
<i>t_cell</i>	7	0,08 %	Ultra-rare

TABLE 4 – Composition du jeu de données Baron après filtrage. Les classes rares correspondent aux types représentant moins de 5 % des cellules, et les classes ultra-rares à ceux représentant moins de 1 %.

Différentes expériences ont été menées afin de répondre à une question simple : est-ce qu'un modèle qui apprend une représentation sur certaines cellules est ensuite capable de rester performant sur des cellules qu'il n'a jamais vues ? Cette question distingue deux cadres d'évaluation. Dans un cadre transductif, toutes les cellules sont disponibles au moment de l'apprentissage et le modèle construit ses groupes à partir de l'ensemble complet. Ce cadre est utile pour explorer un jeu de données déjà acquis, mais il ne dit pas vraiment ce qui se

passerait si l'on ajoutait de nouvelles cellules ensuite. Dans un cadre inductif, au contraire, le modèle est entraîné sur une partie des cellules, puis il doit projeter ou classer de nouvelles cellules sans être réentraîné depuis le début.

Pour rendre cette évaluation possible avec des modèles basés sur des autoencodeurs, nous avons utilisé leur encodeur comme outil de projection. Cette adaptation s'appuie sur une idée déjà présente dans les méthodes de deep clustering comme DEC [33] et IDEC [34], où les cellules sont projetées dans un espace latent avant d'être rapprochées de centres de clusters. L'objectif d'appliquer un modèle entraîné à de nouvelles cellules se rapproche quant à lui davantage de la logique de projection proposée par scArches [35]. Concrètement, une fois le modèle entraîné, ses poids sont gelés : cela signifie qu'ils ne sont plus modifiés. Les cellules d'entraînement sont projetées dans l'espace latent, puis les centres des clusters appris sont calculés dans cet espace. Les nouvelles cellules passent ensuite par le même prétraitement et par le même encodeur ; elles sont finalement rattachées au cluster dont le centre est le plus proche, ou à un groupe de référence construit pendant l'entraînement. Sur Baron, nous avons conservé deux cadres principaux pour l'analyse des méthodes existantes. Le premier est transductif : l'ensemble des 8 569 cellules filtrées est analysé simultanément, puis les performances sont moyennées sur cinq graines aléatoires. Le second est inductif : les cellules sont séparées en ensembles d'entraînement, de validation et de test selon un split stratifié 70/10/20. Les modèles sont ajustés sur 5 997 cellules, les hyperparamètres sont contrôlés sur 858 cellules de validation, puis les performances sont mesurées sur 1 714 cellules de test jamais vues pendant l'entraînement.

Ces deux cadres permettent de mesurer des propriétés complémentaires. Le mode transductif indique si une méthode retrouve correctement la structure globale du jeu de données complet, mais il peut être dominé par les grands types cellulaires et les lots majoritaires. Le mode inductif teste au contraire la capacité à projeter des cellules non vues, dans un contexte où le test contient moins de cellules et où les populations rares deviennent parfois représentées par quelques exemples seulement. Dans chaque cas, les performances ont été évaluées avec des métriques globales, mais aussi avec des métriques centrées sur les petites populations. Nous avons défini les classes rares comme les types cellulaires représentant moins de 5 % des cellules du jeu Baron complet, et les classes ultra-rares comme celles représentant moins de 1 %.

3.2.2 Méthodes

Nous avons alors lancé l'analyse sur un ensemble restreint de méthodes représentatives de l'état de l'art à partir des meilleurs algorithmes de scCluBench [31] : PCA suivie de clustering K-Means ou de Leiden, scDeepCluster, scMAE, scNAME et scCDCG. Les méthodes ont été exécutées avec les recommandations des auteurs lorsque celles-ci étaient disponibles, en conservant les hyperparamètres standards afin d'éviter d'optimiser une méthode au détriment des autres. Pour PCA+Leiden, la résolution du graphe a toutefois été contrôlée afin d'éviter une partition trop grossière : une grille de résolution de 0,1 à 1,0 a été explorée une seule fois sur la première graine, puis la résolution donnant 14 communautés avec le meilleur score de silhouette a été retenue. Cette même résolution a ensuite été conservée pour les cinq graines, même lorsque le nombre final de communautés variait légèrement selon l'initialisation. Ce

choix donne à Leiden la même information a priori sur le nombre attendu de types cellulaires que PCA+KMeans, sans utiliser les annotations biologiques pour choisir les clusters.

Le prétraitement a été identique pour toutes les méthodes afin que les différences observées proviennent principalement des algorithmes et non de la préparation des données. Les cellules et les gènes très peu informatifs ont d’abord été filtrés, puis les comptes ont été normalisés à 10 000 par cellule et transformés par logarithme. Nous avons ensuite conservé les 2 000 gènes hautement variables (*highly variable genes*, HVG) selon la stratégie de Seurat, avant une mise à l’échelle des données. Ce nombre de gènes constitue un compromis standard : il réduit le bruit et le coût de calcul tout en conservant suffisamment d’information pour comparer les méthodes sur Baron.

Dans les expériences avec séparation entraînement/validation/test, la sélection des gènes variables a été réalisée à partir des données d’entraînement afin d’éviter d’utiliser indirectement de l’information provenant du test. Les expériences transductives et inductives ont été répétées cinq fois avec des graines aléatoires différentes. Cette répétition était nécessaire car plusieurs méthodes d’apprentissage profond dépendent de l’initialisation du modèle, et parce que les résultats d’un seul lancement peuvent donner une vision trop optimiste ou trop pessimiste des performances réelles.

Afin de comparer les méthodes de manière homogène, les mêmes métriques ont été calculées pour toutes les expériences. La difficulté principale vient du fait qu’un algorithme de clustering ne prédit pas directement des noms de types cellulaires mais produit seulement des groupes numérotés (par exemple cluster 0, 1 ou 2) dont les labels sont arbitraires.

Nous distinguons deux catégories de métriques pour évaluer les performances, dont les formules mathématiques détaillées sont reportées en Annexe C (Tableau 6) :

Métriques de clustering global (sans alignement) Ces métriques comparent directement la structure de la partition prédite avec la vérité terrain biologique, sans nécessiter d’attribuer un nom ou une classe aux clusters :

- **L’Indice de Rand Ajusté (ARI)** : Évalue la similarité de partitionnement en mesurant la proportion de paires de cellules qui sont regroupées de façon cohérente (soit ensemble, soit séparées) dans la vérité terrain et dans la prédiction, en ajustant le score pour corriger l’accord attendu par hasard. Un score de 1 indique une concordance parfaite.
- **L’Information Mutuelle Normalisée (NMI)** : Fondée sur l’entropie, elle mesure la quantité d’information partagée entre les clusters prédits et les types cellulaires réels. Normalisée entre 0 (indépendance) et 1 (partitions identiques), elle quantifie la réduction d’incertitude sur les annotations grâce à la prédiction.

Métriques de classification (après appariement optimal) Pour calculer des taux de réussite ou d’exactitude par type cellulaire, il faut associer chaque cluster à une étiquette biologique. Nous appliquons l’algorithme hongrois [36] pour résoudre le problème d’affectation linéaire : il détermine l’appariement optimal entre clusters prédits et classes biologiques qui maximise le nombre total de cellules correctement attribuées. Cette approche, couramment adoptée dans des benchmarks comme scCluBench [31], permet ensuite d’évaluer :

- **L’accuracy (ACC)** : Proportion brute de cellules correctement affectées après appariement. Très sensible aux classes majoritaires, elle ne reflète pas la qualité sur les populations minoritaires.
- **La balanced accuracy (Bal ACC)** : Moyenne des rappels par classe biologique, attribuant un poids identique à chaque type cellulaire indépendamment de sa taille.
- **L’accuracy sur les classes rares (RareACC) et ultra-rares (UltraRareACC)** : Même principe que l’accuracy mais spécifique à certaines classes de cellules, les plus rares ($<5\%$) et les ultra-rares ($<1\%$).

3.2.3 Résultats

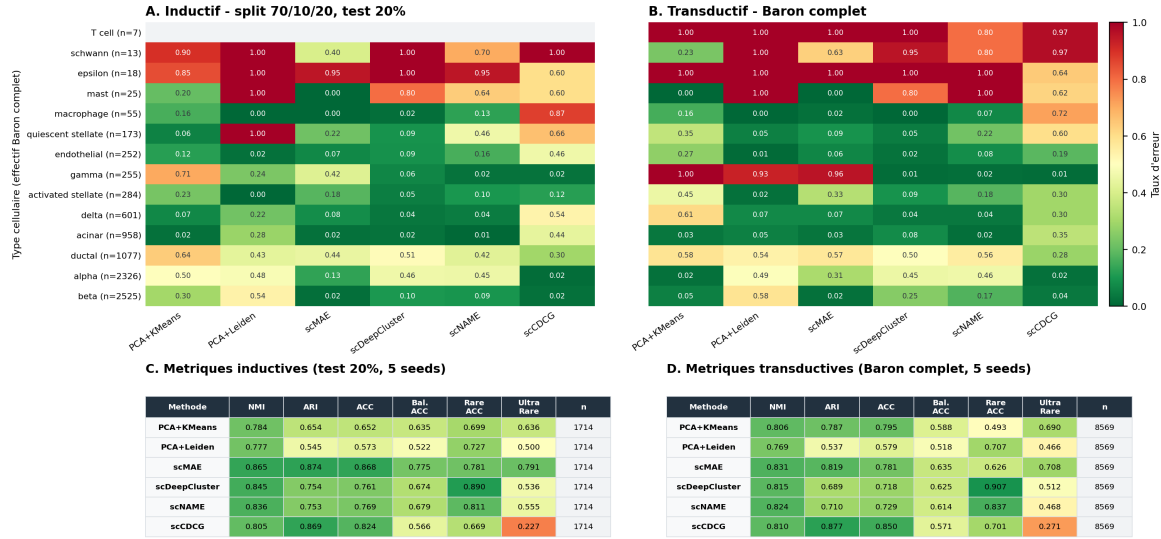
Les résultats obtenus sur Baron montrent que les métriques globales seules ne suffisent pas à caractériser la qualité biologique du clustering. La figure 2 compare deux cadres complémentaires : un cadre inductif, évalué sur le test du split 70/10/20, et un cadre transductif, évalué sur le jeu complet. Le panel présente le taux d’erreur par type cellulaire et par méthode ainsi que les métriques globales associées. Le taux d’erreur est défini comme $1 - \text{rappel}$ après appariement hongrois des clusters.

Dans le cadre transductif, plusieurs méthodes obtiennent des scores globaux élevés. scMAE atteint par exemple le meilleur NMI moyen (0,831), tandis que scCDCG obtient le meilleur ARI (0,877) et la meilleure ACC (0,850). Pourtant, les premières lignes de la heatmap restent très contrastées : les cellules *t_cell*, *schwann*, *epsilon* ou *mast* sont souvent mal retrouvées, alors que les grands types cellulaires comme *alpha*, *beta*, *ductal* ou *acinar* pèsent beaucoup plus fortement dans les scores globaux. Le réglage de PCA+Leiden vers 14 communautés rend la comparaison plus équitable qu’une résolution automatique trop faible, mais ne résout pas le problème : la méthode conserve un NMI de 0,769 et une ARI de 0,537 en transductif, avec des erreurs encore très fortes sur *gamma*, *ductal*, *alpha* et *beta*. Ce contraste explique pourquoi la BalancedACC, la RareACC et l’UltraRareACC sont nécessaires pour interpréter les résultats.

Le mode inductif donne ici des scores globaux plus élevés pour plusieurs méthodes, notamment scMAE, scDeepCluster et scNAME. scMAE obtient ainsi un NMI de 0,865, une ARI de 0,874 et la meilleure BalancedACC du panel (0,775), contre 0,831, 0,819 et 0,635 en transductif. Cette amélioration apparente doit cependant être interprétée prudemment. Le test inductif ne contient que 1 714 cellules, dont 22 cellules ultra-rares : deux *schwann*, quatre *epsilon*, cinq *mast* et onze *macrophage*, tandis que les *t_cell* sont absentes. Dans ces conditions, quelques prédictions correctes ou incorrectes suffisent à faire varier fortement l’UltraRareACC et le taux d’erreur par ligne. L’hypothèse d’un effet de petit effectif ou d’un tirage favorable est donc plausible : un algorithme peut sembler meilleur parce qu’il analyse seulement une ou deux cellules d’un type rare dans le test et les identifie correctement par chance, alors qu’il échouerait plus souvent sur ce même type dans le jeu complet. PCA+Leiden illustre aussi cette limite : malgré la résolution ciblée, ses scores restent modestes en inductif (NMI 0,777, ARI 0,545), et les erreurs demeurent maximales pour *schwann*, *epsilon*, *mast* et *quiescent_stellate*. Cette observation justifie de toujours lire les scores globaux avec les effectifs par type cellulaire, la RareACC, l’UltraRareACC et la stabilité entre graines.

Baron - taux d'erreur par type cellulaire et scores globaux

Erreur = 1 - rappel après appariement hongrois ; résultats moyennés sur 5 seeds. L'inductif utilise un split stratifié 70/10/20, le transductif le jeu complet.



Note : le test inductif contient 1714 cellules ; les types ultra-rares restent très peu représentés (schwann n=2, epsilon n=4, mast n=5, t_cell absent), ce qui rend leur taux d'erreur instable.

FIGURE 2 – Panel de résultats sur le jeu de données Baron. A–B : taux d'erreur par type cellulaire et par méthode, en mode inductif sur le test du split stratifié 70/10/20 et en mode transductif sur le jeu complet. C–D : métriques globales associées. Les deux cadres sont moyennés sur cinq graines. Pour PCA+Leiden, la résolution est fixée à partir d'une recherche visant 14 communautés sur la première graine. Les types cellulaires sont ordonnés par effectif croissant dans le jeu complet. Les scores NMI/ARI peuvent rester corrects alors que l'identification des populations rares dépend fortement du nombre de cellules évaluées.

Ces observations motivent directement le développement de scRAW : l'objectif n'est pas seulement d'améliorer un score moyen, mais de construire une représentation dans laquelle les populations minoritaires conservent assez d'influence pour ne pas être noyées dans les classes dominantes.

4 scRAW

4.1 Positionnement et motivation

scRAW a été développé à partir du constat présenté dans la section précédente : les méthodes de clustering peuvent obtenir de bons scores globaux tout en retrouvant mal les populations cellulaires les plus petites. L'objectif n'est donc pas seulement de construire un nouvel autoencodeur, mais de modifier la manière dont les cellules contribuent à l'apprentissage. Les cellules situées dans de petits groupes ou dans des zones peu denses de l'espace latent reçoivent un poids plus élevé dans la fonction de perte. Elles influencent ainsi davantage la représentation apprise.

La méthode conserve une structure volontairement simple. Elle repose sur un autoencodeur MLP, une pondération dynamique des cellules, une contrainte locale de type triplet loss et une correction adversariale de l'effet de batch lorsque plusieurs lots sont présents. Dans la suite,

la description correspond uniquement à la configuration par défaut de scRAW. Les autres variantes de configuration ne sont pas détaillées ici.

4.2 Pipeline

Le pipeline général de scRAW est présenté dans la figure 3. Nous appliquons aux données brutes les étapes de prétraitement standard, identiques à celles détaillées dans la comparaison de l'existant. De plus, aucune correction de batch séparée n'est appliquée à cette étape ; la prise en compte du batch intervient ensuite directement au sein du modèle.

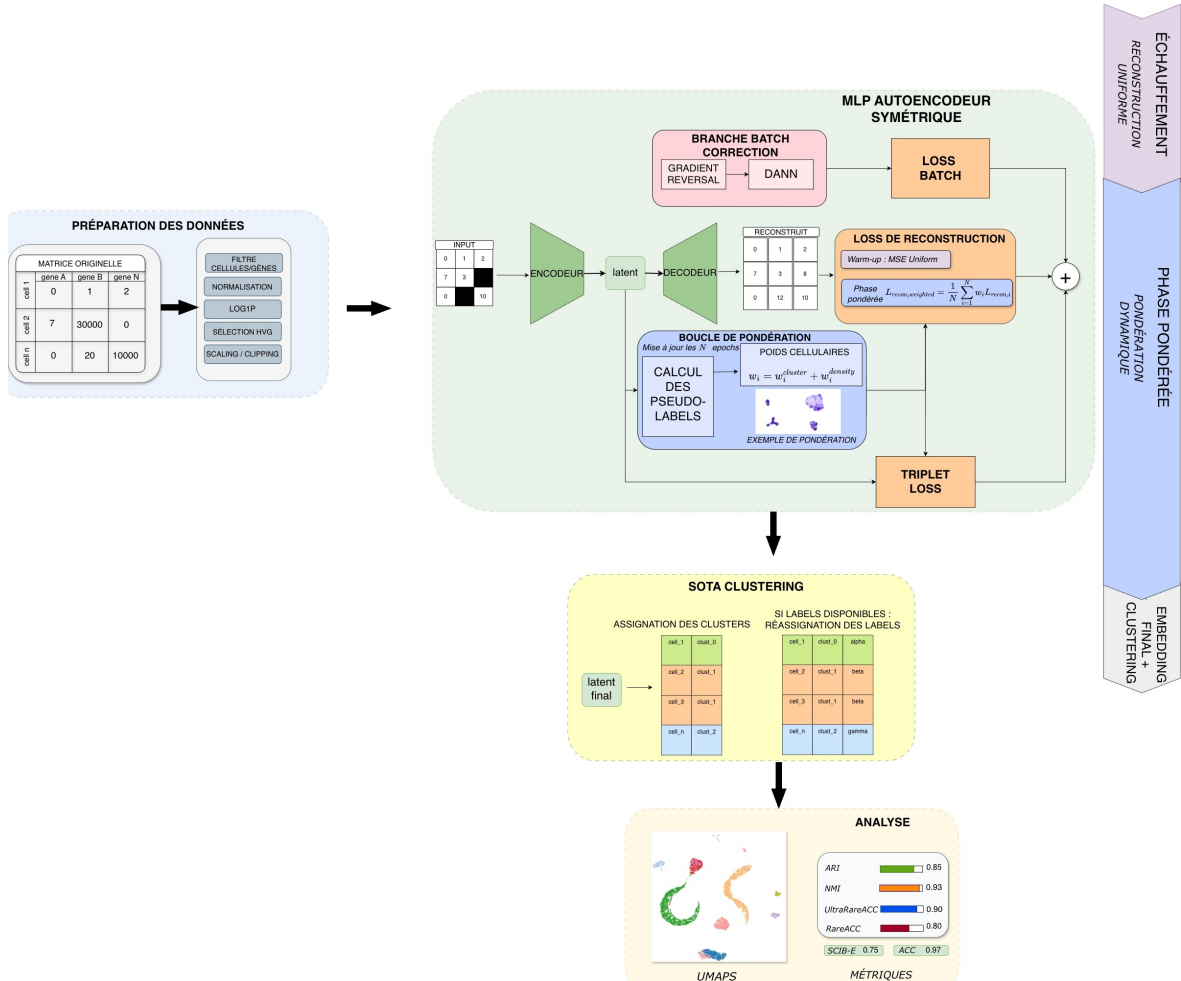


FIGURE 3 – Pipeline général de scRAW. Le modèle apprend un espace latent par autoencodeur, met à jour des poids cellulaires à partir de pseudo-clusters et de la densité locale, puis réalise le clustering final dans l'espace latent.

La matrice prétraitée est ensuite utilisée comme entrée d'un autoencodeur symétrique. Dans la configuration par défaut, l'encodeur comporte trois couches cachées de tailles 512, 256 et 128, puis projette chaque cellule dans un espace latent de dimension 256. Le décodeur suit la structure inverse afin de reconstruire le profil d'expression. Le modèle est entraîné pendant 120 époques, avec une taille de mini-lot de 192, un dropout de 0,3 et un taux d'apprentissage de $1,64 \times 10^{-3}$. Les valeurs complètes sont données en Annexe F.

L'entraînement est divisé en deux phases. Pendant les 55 premières époques, toutes les

cellules contribuent de manière uniforme à la reconstruction. Cette phase sert à stabiliser l'espace latent avant d'introduire la pondération. À partir de l'époque 55, scRAW recalcule périodiquement des pseudo-labels et des poids cellulaires à partir de l'espace latent courant. Dans la configuration par défaut, les pseudo-labels sont obtenus avec Leiden et les poids sont mis à jour toutes les 20 époques. Les poids déjà présents sont lissés par une moyenne mobile, avec un momentum de 0,688, afin d'éviter des changements trop brusques au cours de l'apprentissage.

À la fin de l'entraînement, l'encodeur produit l'embedding final de chaque cellule. Le clustering final est ensuite réalisé avec HDBSCAN dans cet espace latent, en demandant des groupes d'au moins 8 cellules et une estimation locale fondée sur 6 voisins. Les points considérés comme bruit par HDBSCAN ne sont pas réassignés dans cette configuration. Les sorties principales de scRAW sont donc l'espace latent, les labels de clusters prédits, les poids cellulaires finaux, le modèle entraîné, l'historique des pertes et les métriques calculées par le pipeline d'évaluation.

4.3 Fonctions de perte et variantes

Dans la configuration par défaut, l'objectif optimisé combine trois termes de perte :

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon,ponderee}} + \rho(e)\lambda_{\text{triplet}}\mathcal{L}_{\text{triplet}} + \lambda_{\text{batch}}\mathcal{L}_{\text{batch}}.$$

Le premier terme reconstruit les profils d'expression, le deuxième structure localement l'espace latent autour des cellules rares ou isolées, et le troisième réduit l'information de batch visible dans la représentation lorsque plusieurs lots sont présents. Le facteur $\rho(e)$ augmente progressivement le poids effectif du terme triplet au début de son activation.

Reconstruction pondérée

La perte de reconstruction est le terme principal de scRAW. Dans cette configuration, il s'agit d'une erreur quadratique moyenne (MSE) calculée entre la matrice prétraitée X et sa reconstruction $\hat{X}$. Pour une cellule i , on peut l'écrire simplement :

$$\mathcal{L}_{\text{recon},i} = \frac{1}{G} \sum_{g=1}^G (\hat{X}_{i,g} - X_{i,g})^2,$$

où G est le nombre de gènes conservés après prétraitement. Une fraction de 10 % des entrées est masquée aléatoirement pendant l'entraînement, avec une valeur de masquage fixée à 0. Les positions masquées reçoivent un poids interne de 0,8 dans le calcul de la reconstruction. Ce masquage est aussi actif pendant la phase pondérée.

La différence principale avec un autoencodeur standard vient du facteur ω_i , qui module la contribution de chaque cellule :

$$\mathcal{L}_{\text{recon,ponderee}} = \frac{1}{N} \sum_{i=1}^N \omega_i \mathcal{L}_{\text{recon},i}.$$

Pendant la phase initiale, $\omega_i = 1$ pour toutes les cellules. Pendant la phase pondérée, ω_i est recalculé à partir de deux informations. La première est la taille du pseudo-cluster Leiden auquel appartient la cellule : plus le pseudo-cluster est petit, plus le poids augmente. La seconde est la densité locale dans l'espace latent, estimée par la distance au 15^e plus proche voisin : une cellule plus isolée reçoit un poids plus élevé. Dans la configuration par défaut, ces deux composantes sont fusionnées de manière multiplicative :

$$\tilde{\omega}_i = \omega_i^{\text{cluster}} \times \omega_i^{\text{densite}}.$$

Le résultat est normalisé pour avoir une moyenne proche de 1, puis limité entre 0,385 et 10. Cette borne évite qu'une cellule domine entièrement la perte, tout en empêchant les cellules très fréquentes de disparaître de l'apprentissage.

Triplet loss rare-aware

La *triplet loss* est une perte de comparaison relative dans l'espace latent. Elle ne cherche pas à reconstruire directement les profils d'expression ; elle agit plutôt sur l'organisation géométrique des embeddings. Pour un triplet composé d'une cellule ancre a , d'une cellule positive p supposée similaire, et d'une cellule négative n supposée différente, elle pénalise le modèle lorsque l'ancre n'est pas suffisamment plus proche de p que de n . Avec une distance d , une marge m et un ensemble $\mathcal{T}$ de triplets construits dans le mini-batch, on peut l'écrire :

$$\mathcal{L}_{\text{triplet}} = \frac{1}{|\mathcal{T}|} \sum_{(a,p,n) \in \mathcal{T}} \max(0, d(z_a, z_p) - d(z_a, z_n) + m).$$

Ce type de perte est utilisé en apprentissage métrique pour apprendre des représentations où la proximité a un sens, et il a aussi été adapté au clustering scRNA-seq ; par exemple, scHNTL utilise une triplet loss avec des voisins d'ordre élevé pour rapprocher les cellules considérées voisines et éloigner les cellules non voisines [37].

Dans scRAW, ce principe est rendu rare-aware. Le terme est activé à partir de l'époque 60, avec un poids $\lambda_{\text{triplet}} = 0,0501$, une marge de 0,4 et une montée progressive contrôlée par $\rho(e)$. Les ancres sont les cellules dont le poids est supérieur ou égal à 1,2, dans la limite de 64 ancres par mini-batch. Pour chaque ancre, la méthode cherche une cellule positive dans le même pseudo-cluster et une cellule négative dans un autre pseudo-cluster. Cette perte rend donc les groupes rares plus visibles localement dans l'espace latent : elle les rapproche de cellules similaires sans dépendre uniquement du signal global de reconstruction, et les sépare mieux des grands groupes voisins.

Correction adversariale de batch

Le terme $\mathcal{L}_{\text{batch}}$ correspond à la correction adversariale de l'effet de lot. Une petite tête de classification essaie de prédire le batch à partir de l'embedding latent. Grâce à l'inversion de gradient, l'encodeur est poussé à produire une représentation dans laquelle le batch est moins reconnaissable. Dans la configuration par défaut, cette branche est activée si au moins deux batches sont disponibles, avec $\lambda_{\text{batch}} = 0,1176$. Le signal adversarial commence à l'époque 30

et augmente progressivement sur 30 époques.

L’architecture du modèle présenté ici à été sélectionné par une recherche d’hyperparamètres décrite dans la section suivante et une étude d’ablations qui démontre l’importance de chaque élément de l’algorithme dont les résultats sont disponibles en Annexe E.

4.4 Recherche d’hyperparamètres

La configuration par défaut de scRAW a été construite progressivement. L’objectif n’était pas de trouver le meilleur réglage sur un seul jeu de données, mais d’identifier une configuration assez stable pour être utilisée ensuite dans les comparaisons. Les paramètres explorés concernaient principalement l’architecture de l’autoencodeur, le rythme d’entraînement, la pondération des cellules, la triplet loss, la correction adversariale de batch et le clustering final.

Les recherches ont été conduites avec Optuna [38]. Chaque essai correspond à une configuration complète : Optuna propose un ensemble d’hyperparamètres, scRAW est entraîné avec cette configuration, puis les métriques de clustering sont calculées. Le score utilisé sert uniquement à guider la recherche. Il combine des métriques globales et des métriques sensibles aux classes rares, afin d’éviter de sélectionner une configuration qui améliore seulement les grands groupes cellulaires.

La première phase a été une recherche large sur Baron Human Pancreas. Elle a servi à explorer un espace de paramètres volontairement large et à identifier une première famille de configurations pertinentes. Cette étape a donné un point de départ, mais elle ne suffisait pas à définir une configuration finale, car elle restait centrée sur un seul jeu de données.

La deuxième phase a donc évalué les configurations sur plusieurs jeux de données. L’espace de recherche a été réduit à partir des meilleurs comportements observés sur Baron, puis testé sur huit jeux de données d’entraînement. Ces jeux de données, présentés en détail dans l’Annexe B, ont été sélectionnés pour représenter une grande diversité biologique et technique, présentant divers degrés de déséquilibre de classe (indice de Gini) et d’effets de lot (PCR sur PCA). Le score agrégé tenait compte à la fois de la performance moyenne et des datasets les moins bien résolus, afin de favoriser les réglages robustes plutôt que les réglages très bons sur quelques cas seulement.

La dernière phase a encore resserré l’espace de recherche autour des configurations les plus stables. Elle a conservé les choix méthodologiques finalement retenus pour la configuration par défaut : reconstruction MSE, transformation logarithmique, pondération dynamique, triplet loss, correction adversariale de batch et clustering final hybride autour de HDBSCAN. Cette étape a permis de fixer la configuration utilisée dans les expériences du rapport. Les valeurs exactes des hyperparamètres sont données en Annexe F, et les figures d’importance des paramètres sont regroupées en Annexe G.

4.5 Résultats et discussion

L’évaluation finale de scRAW a été conduite sur le même ensemble de huit jeux de données du sous-ensemble commun (détaillé en Annexe B). L’objectif était de vérifier si **scRAW** conservait un bon compromis hors du jeu Baron, puisque la méthode a été conçue sur ce

jeu de données. La comparaison inclut des méthodes orientées vers les populations rares, des méthodes généralistes et des méthodes ou pipelines de correction de batch.

Pour faciliter la lecture, la figure 4 affiche **scRAW** ainsi que les meilleures méthodes observées pour chaque métrique. Les résultats complets, incluant les méthodes non affichées et les variantes avec correction de batch externe, sont fournis en Annexe D. Ces variantes sont conservées comme analyses complémentaires, car elles ne correspondent pas toujours au protocole proposé dans les articles originaux des méthodes comparées.

Top 3 par métrique et par famille + scRAW

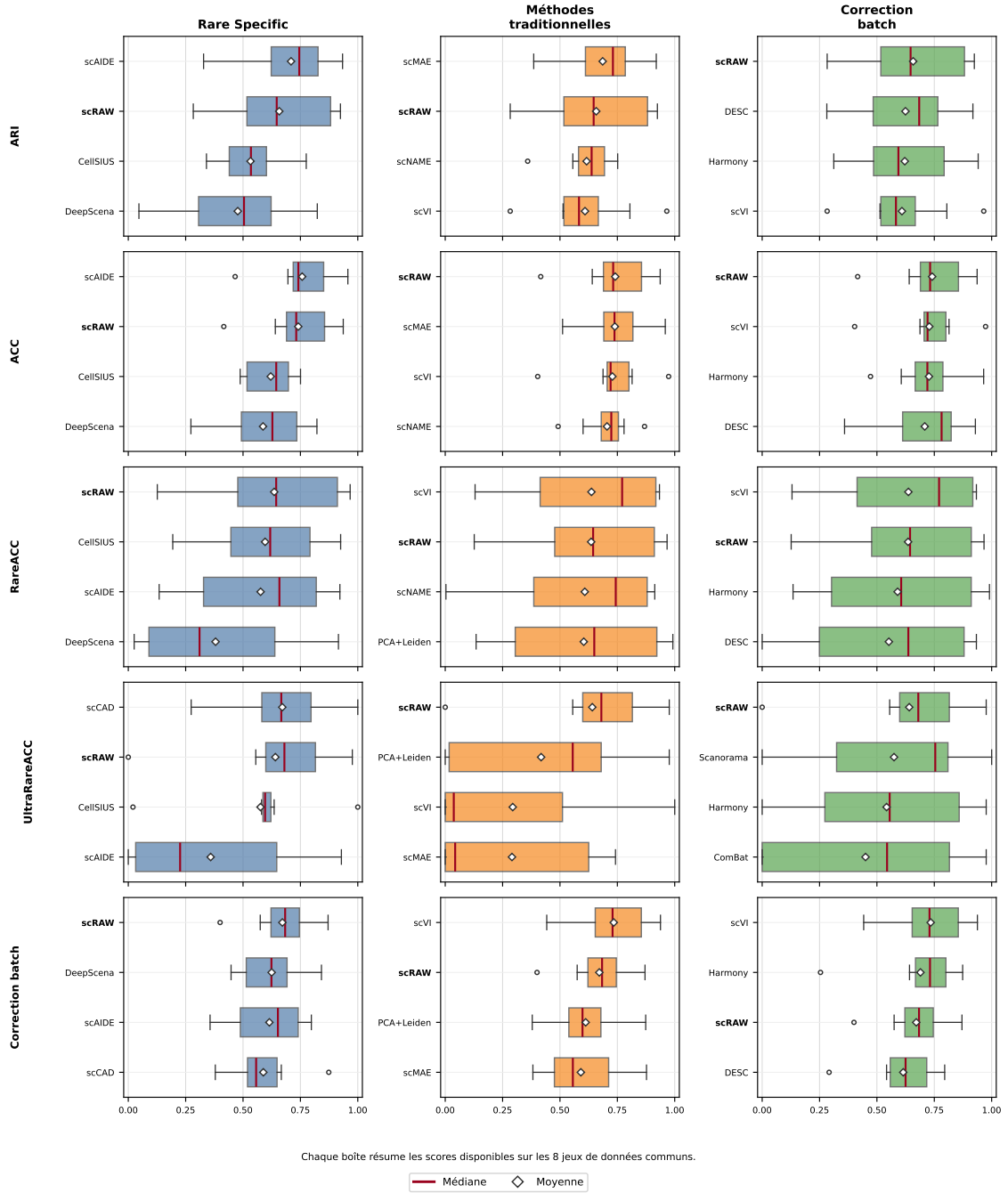


FIGURE 4 – Distribution des performances sur les huit jeux de données communs, par famille de méthodes. Chaque boîte correspond aux scores disponibles sur les jeux de données ; pour l'UltraRareACC, les jeux sans classe ultra-rare ne contribuent pas à la distribution. Les méthodes sont ordonnées par moyenne croissante dans chaque sous-panneau. Chaque sous-panneau affiche **scRAW** et les trois meilleures méthodes de la famille pour la métrique considérée.

La figure 4 montre surtout que les méthodes les plus performantes sur une métrique ne le sont pas nécessairement sur les autres. Certaines méthodes dépassent **scRAW** sur un critère précis, mais elles présentent généralement une baisse plus marquée sur un autre aspect de

l'évaluation.

Par exemple, **scAIDE** et **scAIDE+Harmony** obtiennent de très bons scores globaux en ARI ou en ACC, mais leurs scores sur les populations ultra-rares restent plus faibles que ceux de **scRAW**. À l'inverse, **scCAD** atteint une **UltraRareACC** légèrement supérieure, mais ses scores globaux en ARI et en ACC sont nettement plus bas. Le même type de compromis apparaît pour la correction de batch : **scVI** et **Harmony** corrigent mieux l'effet de batch, mais **scVI** retrouve moins bien les populations ultra-rares, tandis que **Harmony** reste plus faible sur les métriques centrées sur les classes rares.

scRAW n'est donc pas systématiquement la meilleure méthode pour chaque métrique prise séparément. En revanche, ses scores restent élevés sur l'ensemble des critères évalués : 0,658 en ARI, 0,741 en ACC, 0,636 en **RareACC**, 0,641 en **UltraRareACC** et 0,671 pour la correction de batch. Cette stabilité en fait le meilleur compromis observé dans ces expériences pour conserver les populations rares, limiter l'effet de batch et maintenir un clustering global précis.

Les variantes **+Harmony** disponibles en annexe [D](#) confirment cette tendance. Une correction de batch appliquée en post-traitement peut améliorer certaines métriques localement, mais elle ne supprime pas le compromis entre performance globale, correction de batch et préservation des populations rares. **scRAW** intègre cette correction directement dans l'apprentissage, ce qui contribue à l'équilibre obtenu entre les métriques.

Enfin, **scRAW** reste plus coûteux que les pipelines classiques, avec un temps d'exécution moyen d'environ 794 secondes contre quelques secondes pour **PCA+Leiden** ou **Harmony**, mais inférieur à certaines combinaisons profondes avec correction externe comme **scMAE+Harmony**. Ces résultats soutiennent l'hypothèse principale de **scRAW** : augmenter l'influence des cellules rares pendant l'apprentissage permet de mieux préserver les populations minoritaires sans sacrifier fortement la performance globale. La méthode reste cependant à consolider par davantage de répétitions, une analyse plus fine des erreurs par type cellulaire et une validation biologique par marqueurs.

5 Travaux complémentaires

Idée de la section

Regrouper les différentes expériences complémentaires : transfert de la loss et passage à un mode inductif.

5.0 Transfert de loss

- *Présenter l'idée : transférer la loss de reconstruction pondérée ou la logique rare-aware vers d'autres algorithmes de deep learning.*
- *Expliquer pourquoi c'est intéressant : séparer la contribution méthodologique de **scRAW** de son architecture précise.*
- *Décrire le protocole expérimental : méthodes ciblées, variantes avec et sans loss transférée, datasets, métriques.*

- *Montrer les premiers résultats : gains, échecs, stabilité, coût.*
- *Discuter les limites : compatibilité des architectures, difficulté d'intégration, risque de sur-pondération des petites populations.*

5.1 Induction

- *Distinguer transductif et inductif : entraîner sur un ensemble de cellules puis prédire ou projeter de nouvelles cellules non vues.*
- *Décrire le protocole : train sur certains batchs, gel du prétraitement et du modèle, prédiction sur batchs non vus.*
- *Préciser les artefacts sauvegardés : modèle, état du prétraitement, gènes retenus, centroïdes de référence, configuration.*
- *Expliquer pourquoi l'inductif rapproche scRAW d'un usage réel : nouvelles cellules, nouveau batch, réanalyse rapide sans tout réentraîner.*
- *Présenter les résultats comparant inductif et transductif lorsque c'est possible.*
- *Discuter les limites : batch non représentatif, classes absentes du train, baisse possible des performances rares.*

5.2 Interprétation biologique ?

- *À définir*

Conclusion finale à rédiger

- *Résumer les contributions : SCRBenchmark, comparaison standardisée, scRAW, optimisation et premières extensions.*
- *Revenir à la question directrice : préserver les populations rares sans sacrifier entièrement la performance globale.*
- *Présenter les limites : taille des datasets, vérité terrain, sensibilité aux hyperparamètres, coût de calcul, validation biologique.*
- *Ouvrir sur les perspectives : stabilisation de l'inductif, transfert de loss, validation par marqueurs, intégration dans un workflow utilisateur.*

Figures et tableaux restants à préparer

- *Tableau ou figures principal des performances :*
- *UMAP comparatifs :*
- *Matrices de confusion :*
- *Schéma du pipeline scRAW :*

- *Visualisation des poids cellulaires de scRAW :*
- *Figure de recherche d'hyperparamètres : résumé Optuna ou graphiques montrant le compromis entre performance globale, performance rare et stabilité.*
- *Figure inductive :*
- *Tableau du coût de calcul :*
- *Figure ou tableau d'analyse biologique :*

A Structure et organigramme du LaBRI

Le Laboratoire Bordelais de Recherche en Informatique (LaBRI), UMR 5800, au sein duquel s'est déroulé ce stage de fin d'études au sein de l'équipe BKB, est structuré autour de plusieurs grands pôles :

- **Direction** : Pascal Desbarats (Directeur par intérim), Nicolas Hanusse (Directeur adjoint par intérim), Serge Chaumette (Chargé de mission transfert et valorisation).
- **Administration** (Administratrice d'unité : Christine Richard) :
 - *Équipe administrative* (resp. Cathy Roubineau) : Accueil (Claire Douvier, Laurence Pinosa), Assistante administrative (Safia Amsili), Assistante de communication (Auriane Dantès).
 - *Équipe financière* (resp. Chloé Godard) : Gestionnaires (Isabelle Garcia, Anthony Gau, Emmanuelle Lesage, Elia Meyre).
 - *Assistante du SCRIME* : Annick Mersier.
- **Recherche** (structurée en plusieurs départements) :
 - *Image et Son* (Fabien Baldacci)
 - *Combinatoire et Algorithmique* (Cyril Gavaille)
 - *Systèmes et Données* (Jean-Rémy Falleri), département auquel appartient l'équipe BKB.
 - *Méthodes et Modèles formels* (Laurent Bienvenu)
 - *Supports et AlgoriThmes pour les Applications Numériques hAutes performanceS* (Pierre Ramet)
- **Appui à la recherche** :
 - *Ingénieurs affectés sur projet* : Nathalie Furmento, Frédéric Lalanne, Boris Mansencal, Gérald Point.
 - *Équipe soutien "Systèmes - Réseaux"* (resp. Pascal Ung) : Ingénieurs ASR (Michaël Fontaine, Emmanuel Fontaine, Gabriel Larrouy), Technicien ASR (Alexandre Issouli), Ingénieur DEV-Web (Pierre Lacroix).
- **Autre** : Assistant de prévention (Gérald Point).

L'organigramme structurel simplifié correspondant est présenté dans la Figure 5.

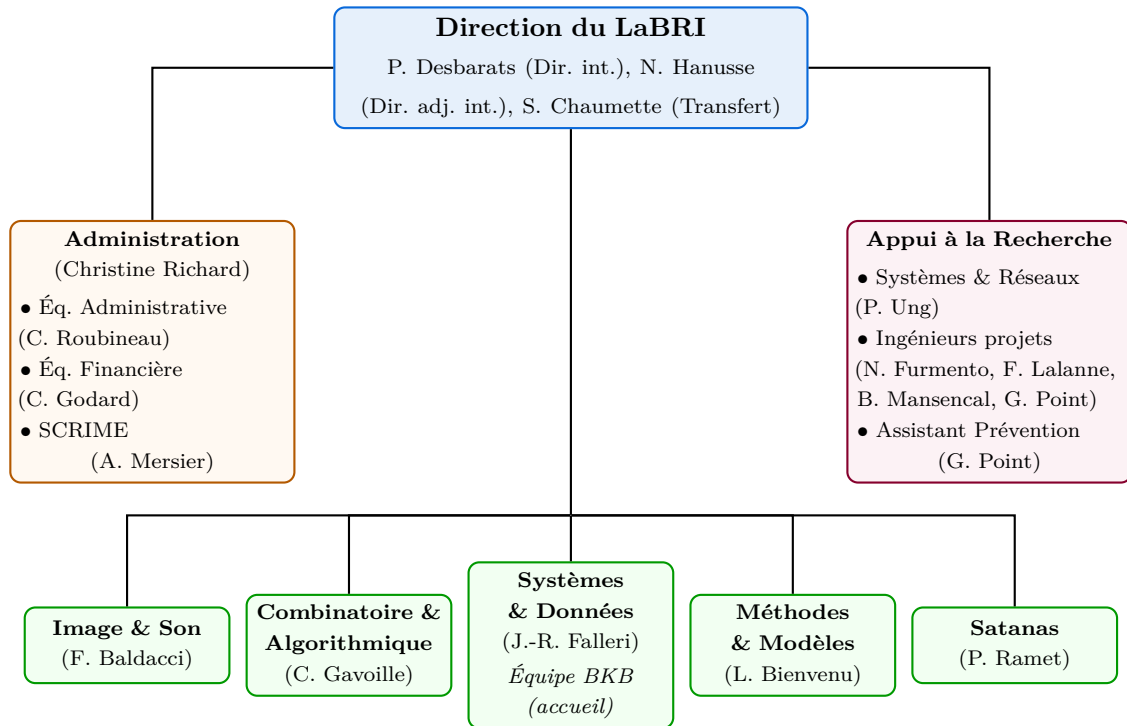


FIGURE 5 – Organigramme structurel simplifié du LaBRI.

B Détails des datasets de l'état de l'art

Cette section fournit les détails des huit jeux de données scRNA-seq du sous-ensemble commun (“common-8”) utilisés dans l'étude, y compris l'espèce, les tailles (cellules et gènes), le nombre de lots (batches), le nombre de populations cellulaires rares, ainsi que leur niveau d'effet de lot (mesuré par PCR sur PCA) et de déséquilibre de classe (mesuré par l'indice de Gini).

Dataset	Espèce	Cellules	Gènes	Lots	Rares (< 5%)	PCR/PCA	Gini
BBAG094 Zeisel	Souris	3 006	19 972	10	4	0,499	0,543
BBAG094 spleen	Souris	9 552	23 341	2	3	0,022	0,644
Baron Human Pancreas	Humain	8 569	20 125	4	9	0,048	0,644
Human testis GSE112013	Humain	6 490	27 477	6	4	0,025	0,304
Kang PBMC	Humain	24 679	35 635	16	1	0,069	0,500
Macaque retina bipolar	Macaque	30 302	36 162	30	4	0,184	0,388
Paul15 bone marrow	Souris	2 730	3 451	1	9	0,000	0,410
Tabula Muris liver	Souris	2 859	22 966	1	5	0,000	0,578

TABLE 5 – Récapitulatif des huit jeux de données du sous-ensemble commun (common-8) utilisés dans nos expériences.

Description des métriques de caractérisation des jeux de données

- **PCR sur PCA (Principal Component Regression on PCA) :** Cette métrique quantifie l'importance globale de l'effet de lot dans un jeu de données en mesurant la part de variance expliquée par la variable de lot (batch) dans l'espace des composantes principales. Une analyse en composantes principales (PCA) est d'abord réalisée sur la matrice d'expression. Pour chaque composante principale, une régression linéaire est ajustée en utilisant le lot comme variable explicative afin d'obtenir un coefficient de détermination R^2 . La métrique finale est la somme de ces R^2 pondérée par la variance expliquée par chaque composante. Un score proche de 0 indique l'absence d'effet de lot, tandis qu'un score proche de 1 indique que l'effet de lot domine la variance des données.
- **Indice de Gini des classes :** Cette métrique mesure le déséquilibre de la distribution des tailles des populations cellulaires (les classes de la vérité terrain) dans le jeu de données. Issu de la théorie économique pour mesurer les inégalités de richesse, cet indice est adapté ici pour mesurer la dispersion statistique des effectifs des différents types cellulaires. Un indice de 0 correspond à une répartition parfaitement équilibrée (toutes les classes ont le même nombre de cellules), tandis qu'un indice proche de 1 traduit un déséquilibre extrême (une classe ultra-majoritaire face à de nombreuses classes très minoritaires).

C Tableau des métriques d'évaluation

Cette section regroupe les définitions, formules mathématiques et interprétations des métriques utilisées pour évaluer et comparer les performances des algorithmes de clustering, tant sur l'aspect global que sur la détection des classes rares et ultra-rares.

Métrique	Rôle	Formule ou principe	Interprétation
ARI	Clustering global	$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]}$, où RI mesure l'accord entre paires de cellules.	Mesure si deux cellules regroupées ensemble dans la vérité biologique le sont aussi dans la prédiction, en corrigeant l'accord attendu au hasard.
NMI	Clustering global	$NMI(Y, \hat{Y}) = \frac{2I(Y; \hat{Y})}{H(Y) + H(\hat{Y})}$	Mesure la quantité d'information partagée entre les annotations biologiques et les groupes prédits.
ACC	Classification après appariement des clusters	$ACC = \frac{1}{n} \max_{\pi} \sum_{i=1}^n \mathbf{1}\{y_i = \pi(\hat{y}_i)\}$	Donne la proportion globale de cellules correctement attribuées après correspondance entre clusters et classes.
BalancedACC	Classification équilibrée	$BalancedACC = \frac{1}{ C } \sum_{c \in C} \frac{TP_c}{N_c}$	Calcule la moyenne des rappels par classe, afin qu'une classe très abondante ne masque pas les erreurs sur les classes petites.
RareACC	Classification des classes rares	$RareACC = \frac{\sum_{c \in \mathcal{R}} TP_c}{\sum_{c \in \mathcal{R}} N_c}$, avec $\mathcal{R} = \{c \mid N_c/n < 5\%\}$	Calcule la proportion brute de cellules correctement affectées parmi l'ensemble des populations rares.
UltraRareACC	Classification des classes ultra-rares	$UltraRareACC = \frac{\sum_{c \in \mathcal{U}} TP_c}{\sum_{c \in \mathcal{U}} N_c}$, avec $\mathcal{U} = \{c \mid N_c/n < 1\%\}$	Calcule la proportion brute de cellules correctement affectées parmi l'ensemble des populations ultra-rares.
Temps	Coût computationnel	$t_{exec} = t_{fin} - t_{debut}$	Permet de comparer les méthodes non seulement sur leur qualité de clustering, mais aussi sur leur coût pratique.

TABLE 6 – Métriques utilisées pour comparer les méthodes. TP_c correspond au nombre de cellules de la classe c correctement attribuées après appariement, et N_c au nombre total de cellules de cette classe.

Dans les formules ci-dessus, y_i désigne l'annotation biologique de la cellule i , $\hat{y}_i$ le cluster prédit par la méthode, n le nombre total de cellules, $\mathcal{C}$ l'ensemble des classes biologiques et π l'appariement optimal obtenu par l'algorithme hongrois.

D Résultats complets sur les huit jeux de données communs

Cette annexe présente les résultats complets utilisés pour construire la figure principale de la section scRAW. Les huit jeux de données correspondent au sous-ensemble commun utilisé pendant l'évaluation généraliste : BBAG094 Zeisel, BBAG094 spleen, Baron Human Pancreas, Human testis GSE112013, Kang PBMC, Macaque retina bipolar, Paul15 bone marrow et Tabula Muris liver. La première figure reprend tous les algorithmes de chaque famille principale, avec **scRAW** ajouté comme repère dans chaque colonne ; la seconde ajoute les expériences complémentaires où Harmony est appliqué en correction de batch.

Résultats complets par famille sur les 8 jeux de données communs

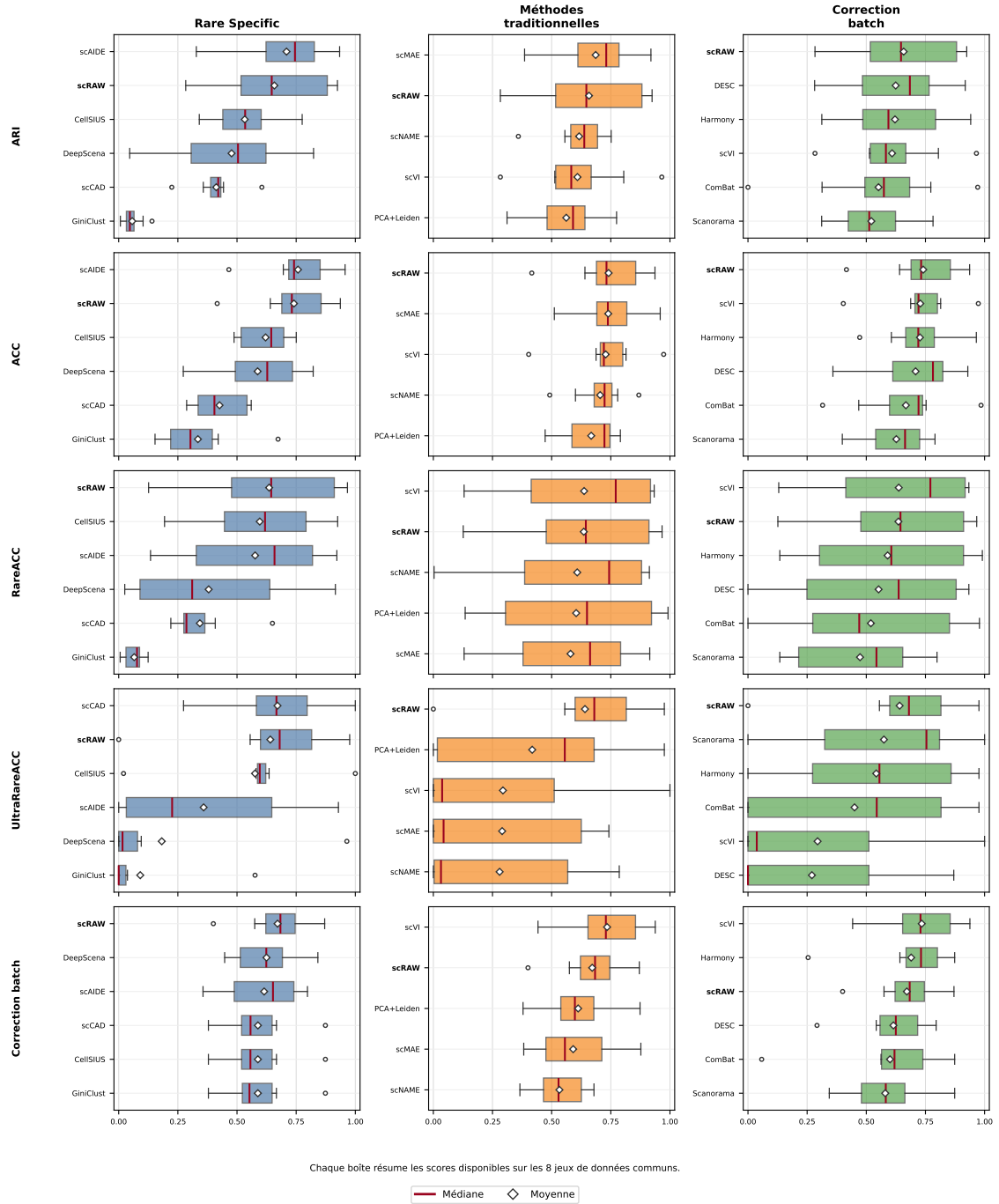


FIGURE 6 – Résultats complets par famille sur les huit jeux de données communs. Les boîtes représentent la distribution des scores disponibles par méthode sur les jeux de données ; pour l'UltraRareACC, les jeux sans classe ultra-rare ne contribuent pas à la distribution. Les méthodes sont ordonnées par moyenne croissante dans chaque sous-panneau. **scRAW** est ajouté comme repère dans chaque famille.

Les tableaux suivants présentent les résultats bruts détaillés par jeu de données pour chaque métrique d'évaluation pour les méthodes principales :

Jeu de données	scRAW	Rare Specific					Traditionnelles				Correction Batch			
	scRAW	scAIDE	CellSIUS	DeepScena	scCAD	GiniClust	scVI	scMAE	PCA+Leiden	scNAME	Harmony	ComBat	DESC	Scanorama
Zeisel	0,430	<u>0,696</u>	0,572	0,316	0,358	0,036	0,513	0,692	0,632	0,645	0,388	0,000	0,752	0,632
Spleen	0,870	0,934	0,465	0,517	0,416	0,141	0,572	<u>0,920</u>	0,433	0,689	0,518	0,580	0,336	0,448
Baron Pancreas	<u>0,916</u>	0,794	0,666	0,572	0,605	0,052	0,621	0,770	0,666	0,708	0,851	0,653	0,918	0,621
Human Testis	0,611	0,611	0,581	0,492	0,399	0,043	0,594	0,651	0,626	0,631	<u>0,652</u>	0,569	0,668	0,471
Kang PBMC	0,546	0,811	0,775	0,773	0,428	0,052	<u>0,805</u>	0,779	0,774	0,751	<u>0,774</u>	0,773	0,533	0,782
Macaque Retina	0,924	0,873	0,497	0,046	0,425	0,023	<u>0,965</u>	0,801	0,497	0,556	0,941	0,971	0,805	0,351
Bone Marrow	0,283	0,328	0,341	0,277	0,224	0,007	0,283	0,386	0,311	<u>0,359</u>	0,312	0,313	0,282	0,311
TM Liver	0,683	0,627	0,365	0,824	0,443	0,103	0,519	0,491	0,554	0,589	0,535	0,554	<u>0,700</u>	0,554
Moyenne	0,658	0,709	0,533	0,477	0,412	0,057	0,609	<u>0,686</u>	0,562	0,616	0,621	0,552	0,624	0,521

TABLE 7 – Résultats bruts de Rand Index Ajusté (ARI) pour les méthodes principales sur les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare Specific					Traditionnelles				Correction Batch			
	scRAW	scAIDE	CellSIUS	DeepScena	scCAD	GiniClust	scVI	scMAE	PCA+Leiden	scNAME	Harmony	ComBat	DESC	Scanorama
Zeisel	0,641	0,835	0,625	0,552	0,343	0,297	0,716	0,832	0,790	0,779	0,606	0,315	<u>0,834</u>	0,790
Spleen	0,829	<u>0,957</u>	0,687	0,729	0,557	0,673	0,796	0,959	0,606	0,869	0,687	0,733	0,518	0,641
Baron Pancreas	0,934	0,750	0,731	0,660	0,537	0,310	0,712	0,731	0,727	0,739	0,891	0,722	<u>0,929</u>	0,687
Human Testis	0,718	0,726	0,664	0,596	0,315	0,217	0,725	0,720	<u>0,744</u>	0,709	0,730	0,642	0,820	0,562
Kang PBMC	0,706	0,732	0,750	0,749	0,408	0,421	0,814	0,744	<u>0,754</u>	0,747	0,753	0,753	0,643	0,744
Macaque Retina	0,937	0,900	0,524	0,273	0,560	0,220	<u>0,973</u>	0,813	0,525	0,601	0,965	0,985	0,806	0,399
Bone Marrow	0,416	0,466	0,488	0,316	0,287	0,153	0,403	0,511	0,473	<u>0,492</u>	0,472	0,468	0,359	0,473
TM Liver	0,746	0,696	0,497	0,822	0,401	0,387	0,688	0,603	0,719	0,706	0,709	0,719	<u>0,758</u>	0,719
Moyenne	<u>0,741</u>	0,758	0,621	0,587	0,426	0,335	0,728	0,739	0,667	0,705	0,727	0,667	0,708	0,627

TABLE 8 – Résultats bruts de Précision Globale (ACC) pour les méthodes principales sur les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare Specific					Traditionnelles				Correction Batch			
	scRAW	scAIDE	CellSIUS	DeepScena	scCAD	GiniClust	scVI	scMAE	PCA+Leiden	scNAME	Harmony	ComBat	DESC	Scanorama
Zeisel	0,611	<u>0,811</u>	0,486	0,735	0,281	0,124	0,627	0,773	0,486	0,876	0,400	0,000	0,497	0,486
Spleen	0,529	0,682	0,694	0,916	0,265	0,079	0,933	0,709	0,992	0,913	<u>0,990</u>	0,978	0,933	0,799
Baron Pancreas	<u>0,896</u>	0,635	0,785	0,606	0,349	0,076	0,918	0,616	0,812	0,893	0,811	0,812	0,776	0,738
Human Testis	0,967	0,844	0,813	0,358	0,291	0,110	0,915	0,847	<u>0,921</u>	0,835	0,906	0,613	0,865	0,599
Kang PBMC	0,678	0,220	0,194	0,263	0,220	0,006	0,158	0,216	<u>0,331</u>	0,003	0,328	0,328	0,289	0,178
Macaque Retina	0,958	0,922	0,925	0,026	0,650	0,038	0,918	0,915	0,925	0,651	0,925	0,969	0,925	0,626
Bone Marrow	0,126	0,364	0,329	0,111	0,279	0,009	0,499	<u>0,434</u>	0,227	0,425	0,227	0,320	0,131	0,227
TM Liver	0,323	0,135	0,543	0,027	<u>0,408</u>	0,081	0,130	<u>0,130</u>	0,135	0,269	0,135	0,135	0,000	0,135
Moyenne	<u>0,636</u>	0,577	0,596	0,380	0,343	0,065	0,637	0,580	0,604	0,608	0,590	0,519	0,552	0,473

TABLE 9 – Résultats bruts de Précision sur Classes Rares (RareACC) pour les méthodes principales sur les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare Specific					Traditionnelles				Correction Batch			
	scRAW	scAIDE	CellSIUS	DeepScena	scCAD	GiniClust	scVI	scMAE	PCA+Leiden	scNAME	Harmony	ComBat	DESC	Scanorama
Zeisel	0,556	<u>0,667</u>	0,593	0,000	0,593	0,037	0,556	0,556	0,556	0,704	0,556	0,000	0,556	0,556
Spleen	<u>0,976</u>	0,000	1,000	0,000	1,000	0,024	1,000	0,000	<u>0,976</u>	0,786	<u>0,976</u>	<u>0,976</u>	0,000	1,000
Baron Pancreas	<u>0,814</u>	0,627	0,636	0,000	0,915	0,576	0,466	0,695	<u>0,797</u>	0,432	<u>0,788</u>	<u>0,788</u>	0,466	0,754
Human Testis	0,643	<u>0,929</u>	0,607	0,964	0,571	0,000	0,000	0,000	0,000	0,000	<u>0,929</u>	0,000	0,000	0,857
Kang PBMC	<u>0,817</u>	<u>0,058</u>	0,596	–	0,274	0,000	0,037	0,043	0,562	0,000	<u>0,540</u>	0,544	0,870	0,093
Macaque Retina	0,680	0,007	0,020	0,095	0,667	0,000	0,000	0,000	0,034	0,007	<u>0,007</u>	0,844	0,000	<u>0,762</u>
Bone Marrow	0,000	0,226	0,581	0,032	<u>0,677</u>	0,000	0,000	0,742	0,000	0,032	0,000	0,000	0,000	0,000
TM Liver	–	–	–	–	–	–	–	–	–	–	–	–	–	–
Moyenne	<u>0,641</u>	0,359	0,576	0,182	0,671	0,091	0,294	0,291	0,418	0,280	0,542	0,450	0,270	0,575

TABLE 10 – Résultats bruts de Précision sur Classes Ultra-Rares (UltraRareACC) pour les méthodes principales sur les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare Specific					Traditionnelles				Correction Batch			
	scRAW	scAIDE	CellSIUS	DeepScena	scCAD	GiniClust	scVI	scMAE	PCA+Leiden	scNAME	Harmony	ComBat	DESC	Scanorama
Zeisel	0,400	0,356	0,379	0,448	0,379	0,379	<u>0,442</u>	0,408	0,379	0,382	0,254	0,058	0,292	0,378
Spleen	0,791	0,690	0,564	0,645	0,565	0,556	0,837	0,551	0,554	0,675	<u>0,811</u>	0,778	0,542	0,514
Baron Pancreas	0,575	0,614	0,550	0,602	0,549	0,550	0,630	0,498	0,548	0,493	0,737	0,566	<u>0,697</u>	0,530
Human Testis	0,730	<u>0,782</u>	0,667	0,775	0,667	0,667	0,775	0,699	0,662	0,679	0,797	0,562	<u>0,778</u>	0,634
Kang PBMC	0,684	0,501	0,523	0,504	0,523	0,526	0,662	0,561	<u>0,726</u>	0,545	<u>0,726</u>	<u>0,726</u>	0,795	<u>0,726</u>
Macaque Retina	0,637	0,451	0,508	0,517	0,508	0,513	0,683	0,382	0,510	0,366	<u>0,677</u>	0,596	0,666	0,344
Bone Marrow	0,683	0,726	0,642	0,664	0,642	0,642	0,908	<u>0,752</u>	0,642	0,513	0,642	0,642	0,564	0,642
TM Liver	0,871	0,798	0,874	0,842	0,874	0,874	0,938	<u>0,877</u>	0,874	0,609	0,874	0,874	0,584	0,874
Moyenne	0,671	0,615	0,588	0,625	0,588	0,588	0,734	0,591	0,612	0,533	<u>0,690</u>	0,600	0,615	0,580

TABLE 11 – Résultats bruts de Correction de Batch pour les méthodes principales sur les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Pour faciliter la lecture des résultats, chaque sous-panneau de la figure 4 affiche **scRAW** ainsi que les trois meilleures méthodes de la famille pour la métrique considérée (basé sur la moyenne des huit datasets). Les méthodes restantes, ainsi que des résultats complémentaires avec méthode de correction de batch associées aux différents algorithmes, sont fournis dans cette annexe. Nous ne présentons pas dans le panneau principal les méthodes couplées à un outil de correction de batch, car les auteurs ne décrivent pas cet usage, ni comme prétraitement recommandé, ni comme composant du modèle. Ces variantes sont donc conservées uniquement comme analyses complémentaires. Pour les méthodes de deep learning généralistes, Harmony a été appliqué sur l’espace latent appris, avant le clustering final, car il s’agissait de la correction de batch la plus robuste dans les expériences de référence [39]. Cette pratique est justifiée par la conception même de Harmony : l’algorithme opère sur tout espace de faible dimension et ne suppose pas que celui-ci provienne d’une PCA obligatoirement. En revanche, aucun des papiers originaux des méthodes benchmarkées ici (scDeepCluster, scMAE, scNAME, scCDCG) ne propose ni n’évalue ce couplage de manière systématique : notre protocole comble ce manque en évaluant l’apport de Harmony sur l’espace latent de plusieurs architectures profondes avec des métriques globales et centrées sur les populations rares.

Résultats complémentaires avec Harmony

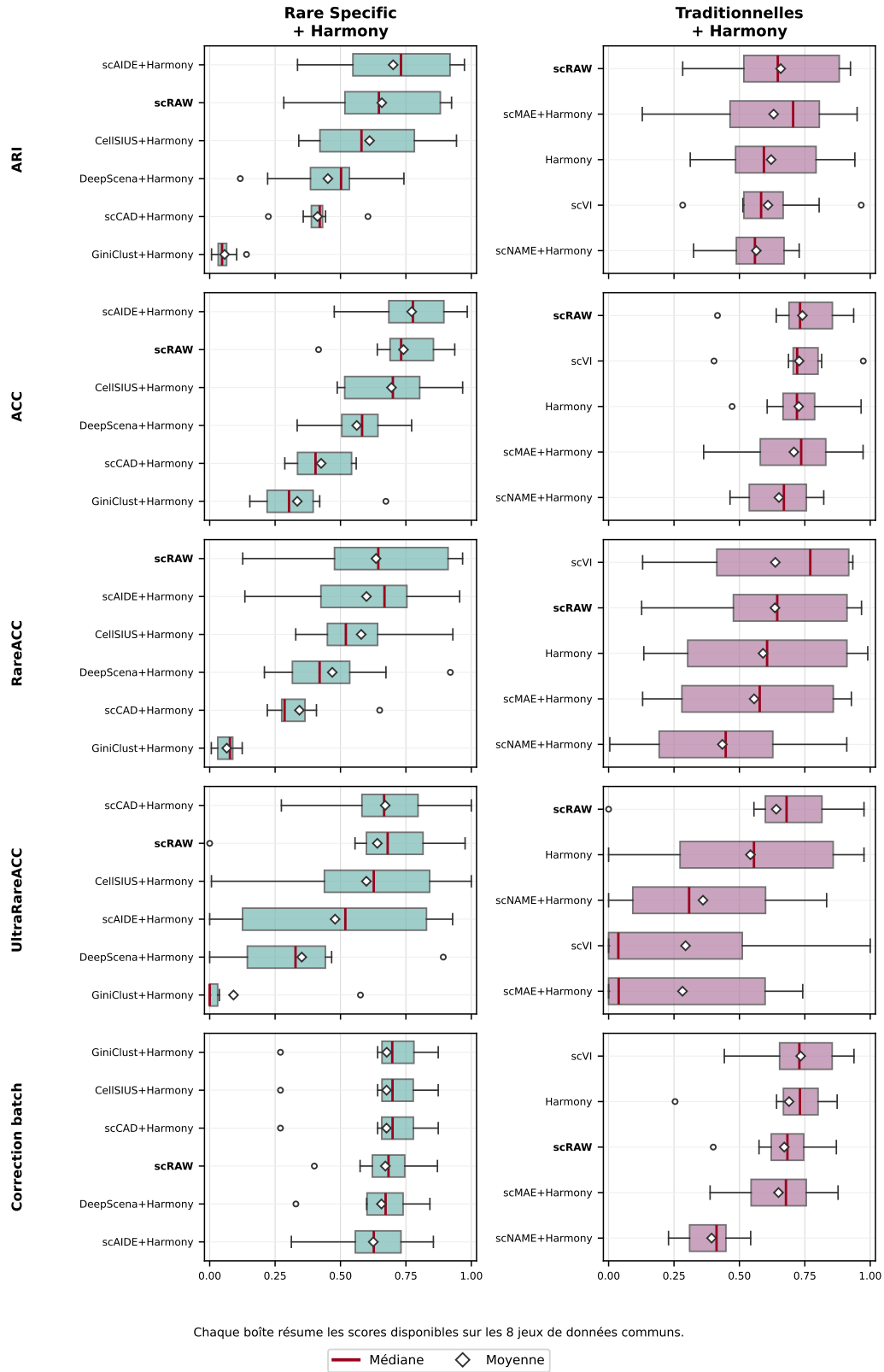


FIGURE 7 – Résultats complémentaires avec Harmony. Les méthodes *Rare Specific*+Harmony sont indiquées à titre exploratoire, car cette association n'est pas documentée comme usage standard par les auteurs. Pour les méthodes de deep learning traditionnelles, Harmony est appliqué sur l'espace latent avant le clustering final. La ligne Harmony correspond au pipeline PCA+Harmony avant clustering ; scVI est conservé comme méthode native intégrant l'information de batch.

Les tableaux suivants présentent les résultats bruts détaillés par jeu de données pour chaque métrique d'évaluation pour les expériences complémentaires avec Harmony :

Jeu de données	scRAW	Rare + Harmony					Trad. + Harmony			
	scRAW	scAIDE	CellSIUS	scCAD	DeepScena	GiniClust	Harmony	scVI	scMAE	scNAME
Zeisel	0,430	0,394	0,440	0,358	0,117	0,036	0,388	<u>0,513</u>	0,129	0,722
Spleen	0,870	0,931	0,558	0,416	0,487	0,141	0,518	<u>0,572</u>	<u>0,911</u>	0,621
Baron Pancreas	0,916	<u>0,914</u>	0,895	0,605	0,440	0,052	0,851	0,621	<u>0,770</u>	0,728
Human Testis	0,611	<u>0,599</u>	0,603	0,399	0,523	0,043	<u>0,652</u>	0,594	0,651	0,653
Kang PBMC	0,546	0,816	0,745	0,428	0,742	0,052	0,774	<u>0,805</u>	0,760	0,492
Macaque Retina	0,924	0,974	0,943	0,425	0,518	0,023	0,941	<u>0,965</u>	0,950	0,498
Bone Marrow	0,283	0,336	<u>0,341</u>	0,224	0,222	0,007	0,312	0,283	0,386	0,325
TM Liver	0,683	<u>0,647</u>	0,365	0,443	0,566	0,103	0,535	0,519	0,491	0,477
Moyenne	<u>0,658</u>	0,701	0,611	0,412	0,452	0,057	0,621	0,609	0,631	0,565

TABLE 12 – Résultats bruts complémentaires de Rand Index Ajusté (ARI) avec Harmony pour les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare + Harmony					Trad. + Harmony			
	scRAW	scAIDE	CellSIUS	scCAD	DeepScena	GiniClust	Harmony	scVI	scMAE	scNAME
Zeisel	0,641	0,625	0,523	0,343	0,363	0,297	0,606	0,716	0,363	0,743
Spleen	0,829	0,959	0,763	0,557	0,679	0,673	0,687	0,796	<u>0,955</u>	0,822
Baron Pancreas	0,934	0,874	<u>0,922</u>	0,537	0,552	0,310	0,891	0,712	0,788	0,738
Human Testis	0,718	0,705	0,669	0,315	0,595	0,217	<u>0,730</u>	0,725	0,724	0,796
Kang PBMC	0,706	0,833	0,731	0,408	0,772	0,421	0,753	<u>0,814</u>	0,748	0,490
Macaque Retina	0,937	0,984	0,967	0,560	0,570	0,220	0,965	<u>0,973</u>	0,973	0,553
Bone Marrow	0,416	0,476	<u>0,488</u>	0,287	0,334	0,153	0,472	0,403	0,511	0,464
TM Liver	0,746	<u>0,720</u>	0,497	0,401	0,631	0,387	0,709	0,688	0,603	0,602
Moyenne	<u>0,741</u>	0,772	0,695	0,426	0,562	0,335	0,727	0,728	0,708	0,651

TABLE 13 – Résultats bruts complémentaires de Précision Globale (ACC) avec Harmony pour les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare + Harmony					Trad. + Harmony			
	scRAW	scAIDE	CellSIUS	scCAD	DeepScena	GiniClust	Harmony	scVI	scMAE	scNAME
Zeisel	0,611	0,649	0,454	0,281	0,324	0,124	0,400	<u>0,627</u>	0,319	0,032
Spleen	0,529	0,734	0,562	0,265	0,920	0,079	0,990	<u>0,933</u>	0,721	0,910
Baron Pancreas	0,896	0,688	0,880	0,349	0,674	0,076	0,811	<u>0,918</u>	0,928	0,561
Human Testis	0,967	0,814	0,498	0,291	0,489	0,110	0,906	<u>0,915</u>	0,841	0,827
Kang PBMC	0,678	0,441	0,438	0,220	<u>0,488</u>	0,006	0,328	0,158	0,164	0,247
Macaque Retina	0,958	<u>0,955</u>	0,929	0,650	<u>0,352</u>	0,038	0,925	0,918	0,913	0,438
Bone Marrow	0,126	<u>0,379</u>	0,329	0,279	0,209	0,009	0,227	0,499	0,434	<u>0,458</u>
TM Liver	0,323	0,135	0,543	<u>0,408</u>	0,291	0,081	0,135	0,130	0,130	0,004
Moyenne	<u>0,636</u>	0,599	0,579	0,343	0,468	0,065	0,590	0,637	0,556	0,435

TABLE 14 – Résultats bruts complémentaires de Précision sur Classes Rares (RareACC) avec Harmony pour les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare + Harmony					Trad. + Harmony			
	scRAW	scAIDE	CellSIUS	scCAD	DeepScena	GiniClust	Harmony	scVI	scMAE	scNAME
Zeisel	<u>0,556</u>	0,519	0,296	0,593	0,370	0,037	<u>0,556</u>	<u>0,556</u>	0,519	0,185
Spleen	<u>0,976</u>	0,000	1,000	1,000	0,286	0,024	<u>0,976</u>	1,000	0,000	0,833
Baron Pancreas	<u>0,814</u>	0,780	0,627	0,915	0,466	0,576	<u>0,788</u>	0,466	0,678	0,458
Human Testis	<u>0,643</u>	0,929	<u>0,893</u>	0,571	<u>0,893</u>	0,000	0,929	0,000	0,000	0,000
Kang PBMC	0,817	0,058	<u>0,788</u>	0,274	–	0,000	0,540	0,037	0,038	0,308
Macaque Retina	<u>0,680</u>	0,878	0,007	0,667	0,000	0,000	0,007	0,000	0,000	0,000
Bone Marrow	0,000	0,194	0,581	<u>0,677</u>	0,097	0,000	0,000	0,000	0,742	0,742
TM Liver	–	–	–	–	–	–	–	–	–	–
Moyenne	<u>0,641</u>	0,479	0,599	0,671	0,352	0,091	0,542	0,294	0,282	0,361

TABLE 15 – Résultats bruts complémentaires de Précision sur Classes Ultra-Rares (UltraRareACC) avec Harmony pour les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare + Harmony					Trad. + Harmony			
	scRAW	scAIDE	CellSIUS	scCAD	DeepScena	GiniClust	Harmony	scVI	scMAE	scNAME
Zeisel	0,400	0,312	0,270	0,270	0,329	0,270	0,254	0,442	0,388	<u>0,402</u>
Spleen	0,791	0,756	0,785	0,783	<u>0,822</u>	0,790	0,811	0,837	0,767	0,310
Baron Pancreas	0,575	0,562	0,730	0,729	0,681	<u>0,730</u>	0,737	0,630	0,604	0,309
Human Testis	0,730	0,723	0,776	0,777	0,712	<u>0,777</u>	0,797	0,775	0,752	0,430
Kang PBMC	<u>0,684</u>	0,573	0,664	0,663	0,602	0,664	0,726	0,662	0,561	0,423
Macaque Retina	0,637	0,540	0,668	0,669	0,600	0,666	<u>0,677</u>	0,683	0,495	0,229
Bone Marrow	0,683	0,682	0,642	0,642	0,664	0,642	0,642	0,908	<u>0,752</u>	0,543
TM Liver	0,871	0,855	0,874	0,874	0,842	0,874	0,874	0,938	<u>0,877</u>	0,504
Moyenne	0,671	0,625	0,676	0,676	0,656	0,677	<u>0,690</u>	0,734	0,649	0,394

TABLE 16 – Résultats bruts complémentaires de Correction de Batch avec Harmony pour les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Ces résultats détaillés permettent d’analyser le comportement individuel de chaque méthode sur chacun des jeux de données, offrant ainsi une vision plus fine et contrastée que les seules moyennes globales.

E Étude d’ablation de scRAW

Cette section présente l’étude d’ablation menée pour valider l’apport individuel de chaque composante de **scRAW** (la pondération dynamique, la perte triplet rare-aware et la correction adversariale de batch DANN). Toutes les variantes ont été entraînées sur le jeu de données Baron Human Pancreas en utilisant HDBSCAN comme algorithme de clustering final.

Le Tableau 17 détaille les scores obtenus pour les variantes suivantes :

- **MSE** : Autoencodeur classique avec une perte MSE standard sans pondération, sans triplet loss et sans correction adversariale.
- **Weighted MSE** : Variante incluant la pondération dynamique des cellules par taille de pseudo-cluster et densité locale, mais sans triplet loss ni correction adversariale.
- **Weighted + triplet** : Autoencodeur pondéré avec triplet loss rare-aware, sans correction de batch adversariale.
- **Weighted + DANN** : Autoencodeur pondéré avec correction adversariale (DANN), sans triplet loss.
- **Full (scRAW)** : Modèle complet associant la pondération dynamique, la triplet loss et la branche adversariale.
- **Full single update** : Modèle scRAW complet, mais où la mise à jour des poids cellulaires n’est effectuée qu’une seule fois au lieu d’être actualisée périodiquement.
- **Full density-only** : Modèle scRAW complet, mais où la pondération des cellules repose uniquement sur la densité locale (la composante liée à la taille des pseudo-clusters est désactivée).
- **Full cluster-size-only** : Modèle scRAW complet, mais où la pondération repose uniquement sur la taille des pseudo-clusters Leiden (la composante liée à la densité locale est désactivée).

Variante	ARI	NMI	ACC	B. ACC	Rare	U. Rare	scIB-E	Temps (s)
MSE	0,189	0,397	0,407	0,427	0,350	0,695	0,427	70,8
Weighted MSE	0,269	0,455	0,456	0,520	0,390	0,686	0,426	518,2
Weighted + triplet	0,851	0,871	0,885	0,813	0,809	0,805	0,431	454,9
Weighted + DANN	0,246	0,560	0,545	0,611	0,491	0,653	0,574	279,7
Full (scRAW)	0,916	0,884	0,934	0,870	0,896	0,814	0,590	337,9
Full single update	0,900	0,873	0,913	0,747	0,793	0,686	0,599	285,9
Full density-only	0,920	0,888	0,926	0,753	0,796	0,703	0,587	334,4
Full cluster-size-only	0,723	0,777	0,800	0,869	0,897	0,949	0,596	347,2

TABLE 17 – Résultats de l’étude d’ablation des composantes de scRAW sur le jeu de données Baron Human Pancreas (clustering final HDBSCAN). B. ACC : BalancedACC; Rare : RareACC; U. Rare : UltraRareACC; scIB-E : score d’intégration de batch scIB (entropie de mélange).

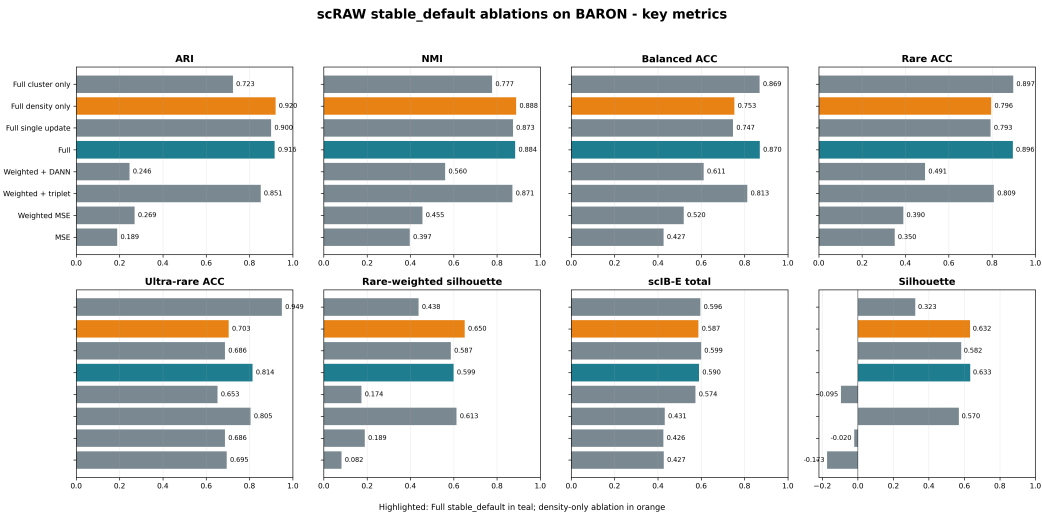


FIGURE 8 – Comparaison graphique des performances des variantes d’ablation de scRAW pour les métriques de clustering global (ARI, NMI, ACC), la détection des classes rares (RareACC, UltraRareACC) et le score d’intégration de batch (scIB-E).

Discussion des résultats

L’analyse de ces résultats d’ablation permet de tirer plusieurs enseignements essentiels sur l’architecture de scRAW :

- **Importance de la pondération de la reconstruction** : L’autoencodeur classique (*MSE*) obtient des scores très limités ($ARI = 0,189$ et $NMI = 0,397$). L’activation de la

pondération dynamique (*Weighted MSE*) augmente nettement ces scores ($\text{ARI} = 0,269$ et $\text{NMI} = 0,455$) ainsi que la BalancedACC (0,520 contre 0,427), ce qui confirme que moduler l'importance des cellules selon leur rareté et leur densité locale aide à structurer l'espace latent de manière plus équitable.

- **Rôle fondamental de la triplet loss rare-aware** : L'ajout de la perte triplet (*Weighted + triplet*) engendre une amélioration spectaculaire sur l'ensemble des critères. L'ARI bondit à 0,851, la BalancedACC à 0,813, et les exactitudes sur les populations rares et ultra-rares dépassent 0,80. Cela montre que guider explicitement l'organisation géométrique des embeddings en rapprochant les cellules similaires et en isolant les petits groupes est indispensable pour obtenir une partition biologique claire et robuste.
- **Synergie avec la correction adversariale** : L'utilisation de DANN seul avec la pondération (*Weighted + DANN*) améliore considérablement le score de correction de batch (l'entropie scIB-E grimpe à 0,574) mais reste inefficace pour former des groupes biologiquement cohérents sans le terme de triplet loss. Le modèle complet (*Full*) réunit le meilleur des deux aspects : il maintient une performance de clustering très élevée ($\text{ARI} = 0,916$, $\text{ACC} = 0,934$) tout en garantissant un excellent niveau d'intégration de batch (scIB-E = 0,590).
- **Rythme de mise à jour des poids** : Dans la variante *Full single update*, recalculer les poids une seule fois plutôt que périodiquement dégrade la détection des populations les plus petites (l'Ultra-rare ACC chute de 0,814 à 0,686). L'actualisation continue des poids au fil de l'apprentissage est donc cruciale pour s'adapter à l'évolution de la topologie de l'espace latent.
- **Complémentarité des composantes de pondération** : Se restreindre à la seule densité locale (*Full density-only*) offre la meilleure ARI globale (0,920) mais diminue la préservation des classes ultra-rares (Ultra-rare ACC = 0,703). Inversement, la pondération par taille de pseudo-cluster seule (*Full cluster-size-only*) favorise massivement les populations très petites (Ultra-rare ACC = 0,949) mais dégrade le partitionnement global ($\text{ARI} = 0,723$). La combinaison multiplicative des deux termes (modèle complet *Full*) permet d'atteindre le compromis le plus équilibré et le plus robuste.

F Hyperparamètres de la configuration de référence scRAW

Cette annexe détaille les hyperparamètres effectivement actifs dans la configuration par défaut utilisée pour décrire scRAW dans le rapport.

TABLE 18 – Hyperparamètres actifs du preset `stable_default` de scRAW.

Bloc	Paramètre	Valeur	Rôle
Prétraitement	<code>n_top_genes</code>	2000	Nombre de gènes hautement variables conservés.
Prétraitement	<code>min_genes_per_cell</code>	200	Filtrage des cellules de mauvaise qualité.
Prétraitement	<code>min_cells_per_gene</code>	3	Filtrage des gènes très peu détectés.
Prétraitement	<code>target_sum</code>	20000	Normalisation des comptes par cellule.
Prétraitement	<code>scale_max_value</code>	10.0	Seuil appliqué après standardisation.
Prétraitement	<code>hvg_flavor</code>	<code>seurat</code>	Méthode de sélection des HVG.
Architecture	<code>hidden_layers</code>	512,256,128	Tailles des couches cachées de l'encodeur.
Architecture	<code>z_dim</code>	256	Dimension de l'espace latent.
Architecture	<code>dropout</code>	0.3	Dropout dans l'encodeur et le décodeur.
Entraînement	<code>epochs</code>	120	Nombre total d'époques.
Entraînement	<code>warmup_epochs</code>	55	Phase sans pondération dynamique.
Entraînement	<code>lr</code>	0.00164076083297036	Taux d'apprentissage initial.
Entraînement	<code>batch_size</code>	192	Taille des mini-batches.
Entraînement	<code>seed</code>	64	Graine utilisée pour l'entraînement.
Reconstruction	<code>reconstruction_distribution</code>	<code>mse</code>	Perte de reconstruction utilisée.
Reconstruction	<code>masking_rate</code>	0.1	Fraction d'entrées masquées aléatoirement.
Reconstruction	<code>masking_value</code>	0.0	Valeur utilisée pour les entrées masquées.
Reconstruction	<code>masked_recon_weight</code>	0.8	Poids interne des positions masquées.
Reconstruction	<code>masking_apply_weighted</code>	<code>True</code>	Masquage aussi actif pendant la phase pondérée.
Pondération	<code>weight_exponent</code>	0.2	Intensité de la pondération par taille de pseudo-cluster.
Pondération	<code>density_knn_k</code>	15	Voisinage utilisé pour estimer la densité locale.
Pondération	<code>density_weight_exponent</code>	1.0	Exposant de la composante densité.
Pondération	<code>density_weight_clip</code>	3.0	Borne supérieure avant normalisation de la densité.
Pondération	<code>weight_fusion_mode</code>	<code>multiplicative</code>	Fusion des composantes cluster et densité.
Pondération	<code>cluster_weight_power</code>	1.0	Puissance appliquée à la composante cluster.
Pondération	<code>density_weight_power</code>	1.0	Puissance appliquée à la composante densité.
Pondération	<code>dynamic_weight_momentum</code>	0.6884621079434989	Lissage des poids entre deux mises à jour.
Pondération	<code>dynamic_weight_update_interval</code>	20	Fréquence de recalcul des poids.

Suite page suivante

Bloc	Paramètre	Valeur	Rôle
Pondération	min_cell_weight	0.3845423008053828	Borne inférieure du poids cellulaire final.
Pondération	max_cell_weight	10.0	Borne supérieure du poids cellulaire final.
Pseudo-labels	pseudo_label_method	leiden	Méthode utilisée pour les pseudo-clusters.
Pseudo-labels	pseudo_leiden_resolution_strategy	scraw_default	Recherche de résolution Leiden utilisée par scRAW.
Pseudo-labels	unsupervised_k_selection	heuristic	Estimation heuristique de K si nécessaire.
Pseudo-labels	unsupervised_k_min	8	Borne minimale de K .
Pseudo-labels	unsupervised_k_max	30	Borne maximale de K .
Triplet	rare_triplet_weight	0.05007581780188212	Poids de la triplet loss.
Triplet	rare_triplet_start_epoch	60	Début de la triplet loss.
Triplet	rare_triplet_margin	0.4	Marge de séparation.
Triplet	rare_triplet_min_weight	1.2	Poids minimal pour choisir une ancre.
Triplet	max_triplet_anchors_per_batch	64	Nombre maximal d'ancres par mini-batch.
Batch	use_batch_conditioning	True	Activation de la branche batch si plusieurs lots existent.
Batch	batch_correction_key	batch	Colonne de métadonnées utilisée pour le batch.
Batch	adversarial_batch_weight	0.11763398875166495	Poids de la perte adversariale batch.
Batch	adversarial_lambda	1.0	Intensité de l'inversion de gradient.
Batch	adversarial_start_epoch	30	Début de la branche adversariale.
Batch	adversarial_ramp_epochs	30	Durée de montée progressive du signal adversarial.
Clustering final	hdbscan_min_cluster_size	8	Taille minimale d'un cluster HDBSCAN.
Clustering final	hdbscan_min_samples	6	Nombre minimal d'échantillons HDBSCAN.
Clustering final	hdbscan_cluster_selection_method	leiden	Stratégie de sélection des clusters.
Clustering final	hdbscan_reassign_noise	False	Pas de réassignation des points bruit.

G Figures de recherche d'hyperparamètres scRAW

Cette annexe regroupe les sorties Optuna d'importance moyenne des hyperparamètres pour les trois recherches utilisées dans la construction de `stable_default`.

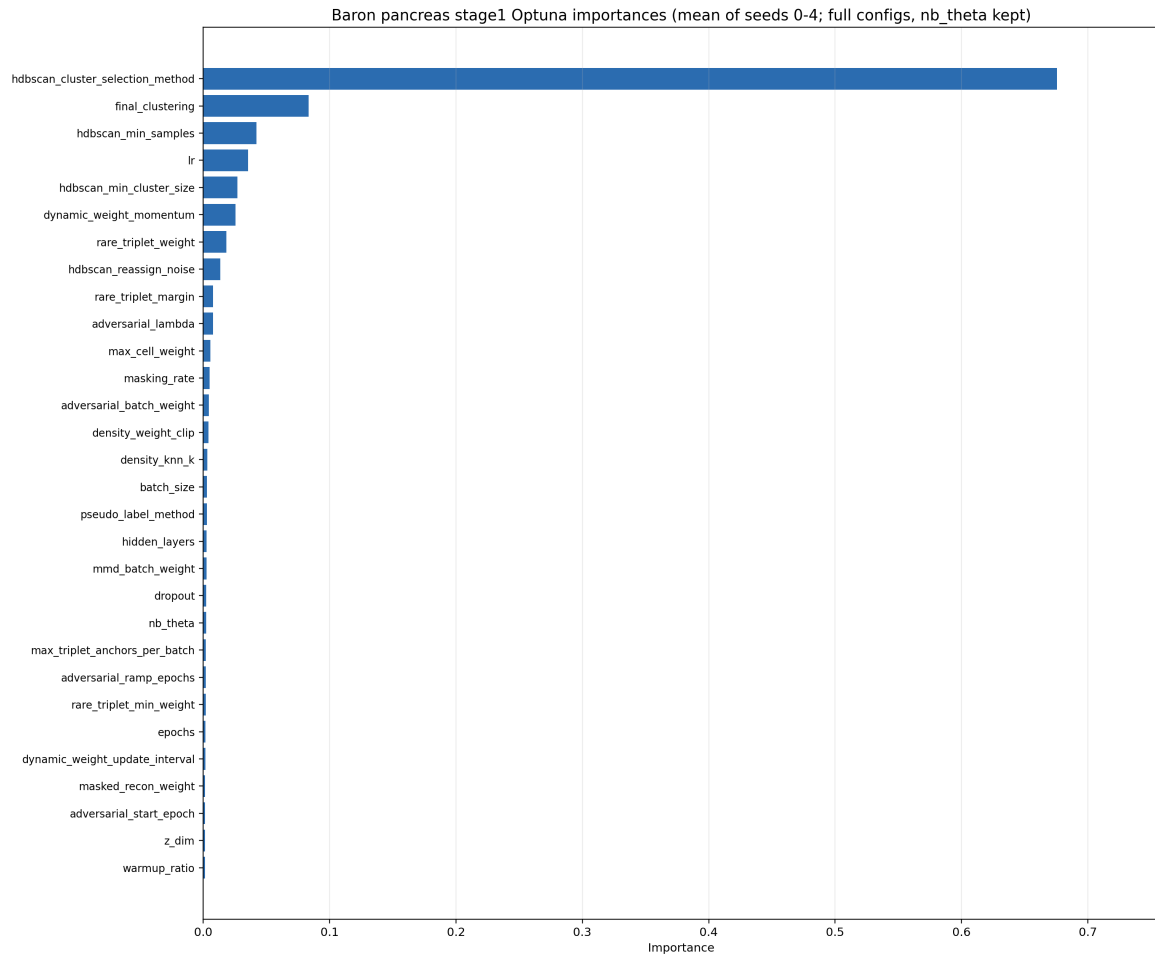


FIGURE 9 – Importance des hyperparamètres pour la recherche large sur Baron Human Pancreas.

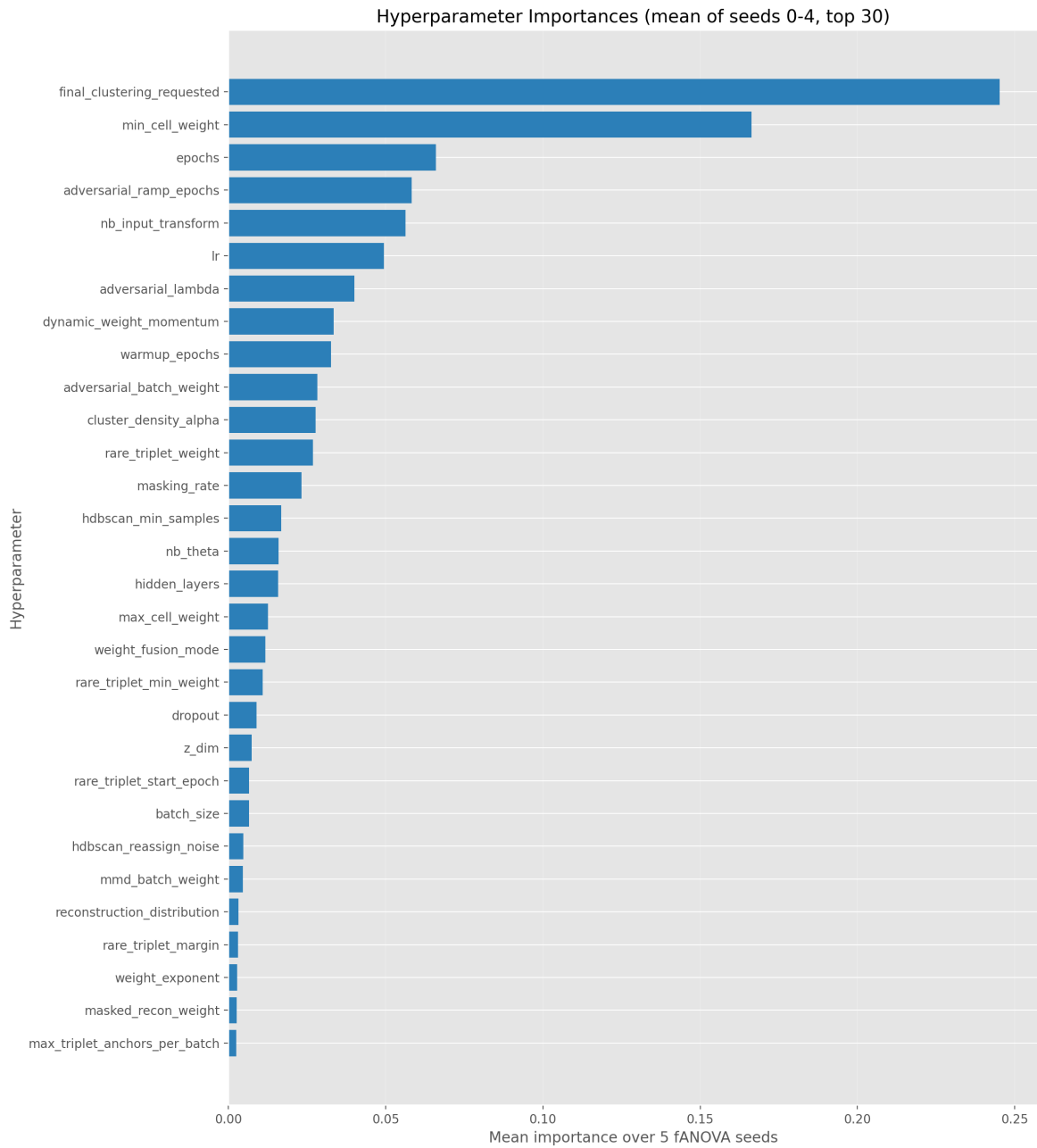


FIGURE 10 – Importance des hyperparamètres pour la recherche généraliste sur huit jeux de données.

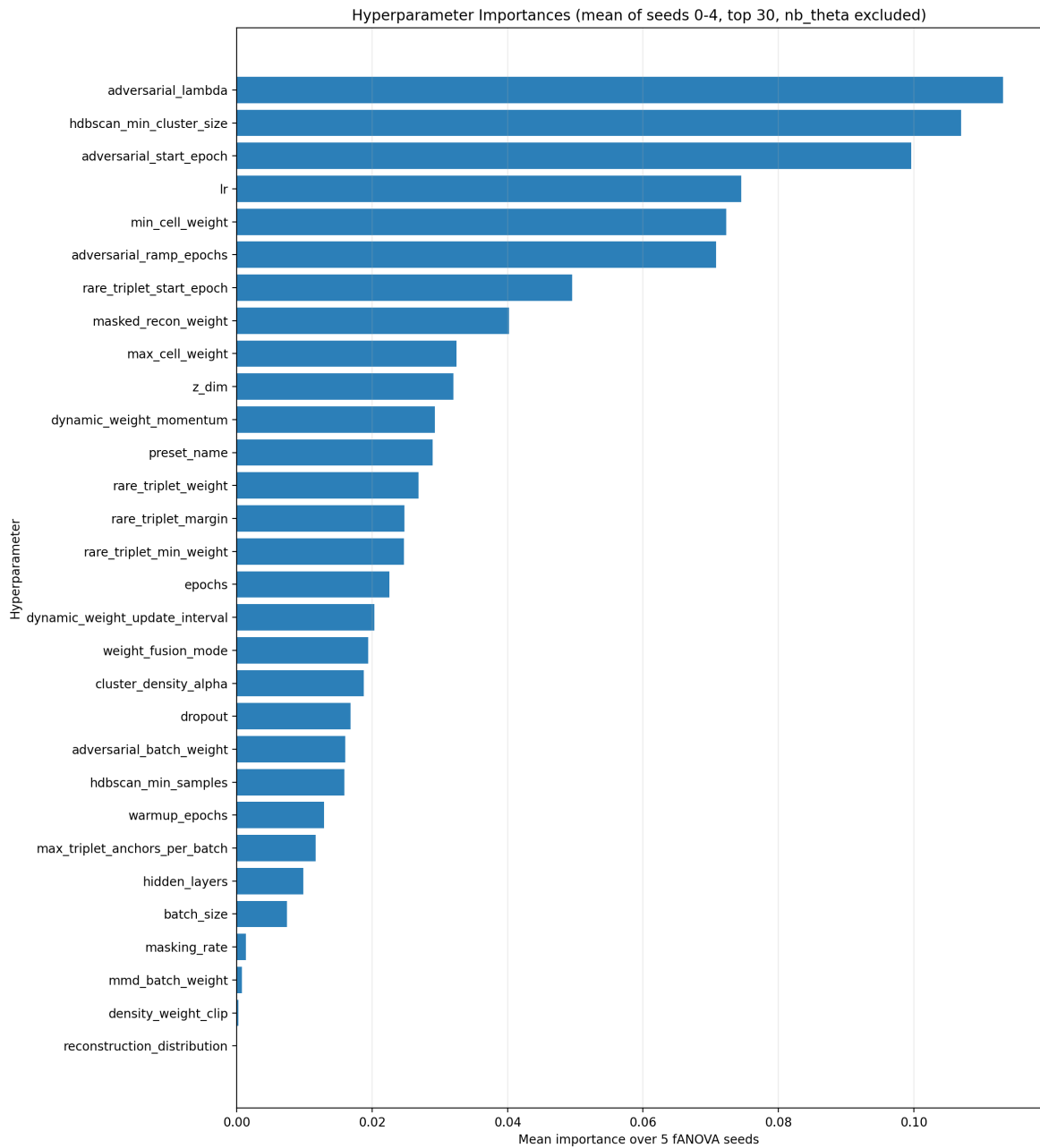
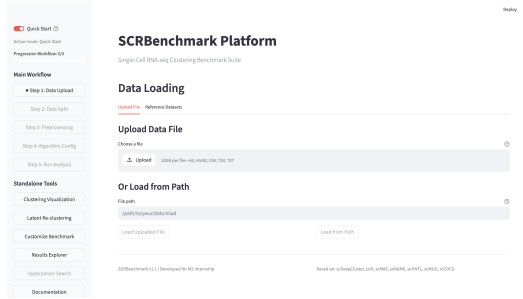


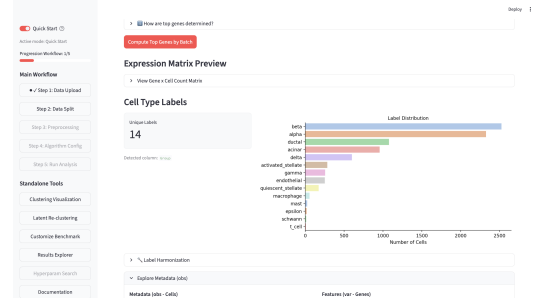
FIGURE 11 – Importance des hyperparamètres pour la recherche restreinte `stable_generalist`.

H Screenshots de l'interface graphique SCRBenchmark

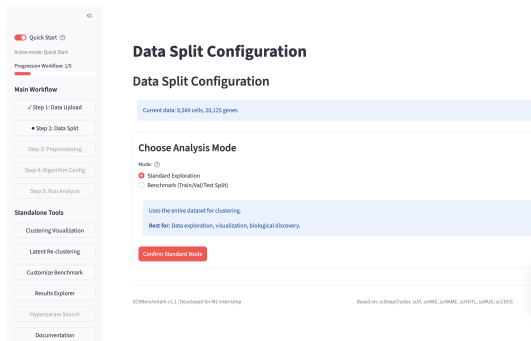
Cette section regroupe les captures d'écran illustrant le fonctionnement et le flux complet de l'interface utilisateur Streamlit développée pour SCRBenchmark.



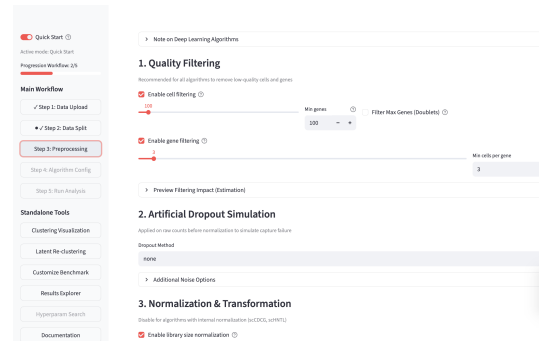
(a) Importation des données



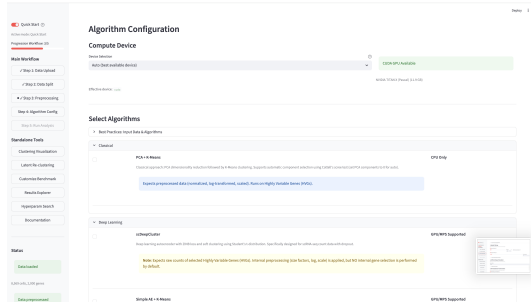
(b) Visualisation des données



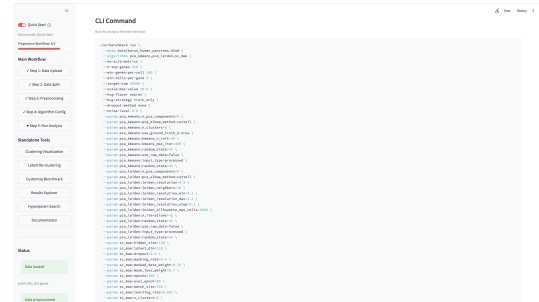
(c) Choix de l'expérience



(d) Prétraitement

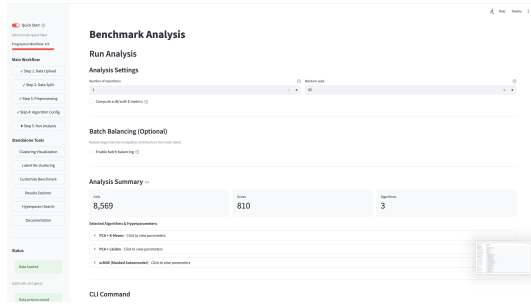


(e) Choix des algorithmes



(f) Génération CLI

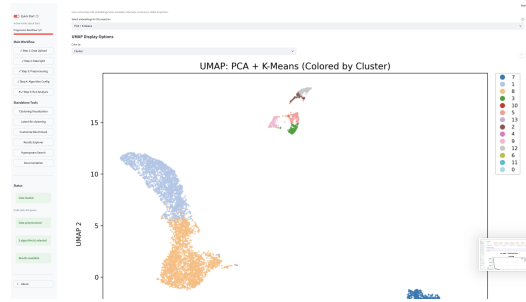
FIGURE 12 – Aperçu des différentes étapes de l'interface Streamlit de SCRBenchmark (1^{re} partie : configuration et préparation).



(g) Exécution des calculs



(h) Résultats des métriques



(i) UMAPs des résultats

FIGURE 12 – Aperçu des différentes étapes de l'interface Streamlit de SCRBenchmark (2^e partie : exécution et résultats).

Bibliographie

- [1] Dominic GRÜN et al. « Single-cell messenger RNA sequencing reveals rare intestinal cell types ». In : *Nature* 525.7568 (2015), p. 251-255. DOI : [10.1038/nature14966](https://doi.org/10.1038/nature14966).
- [2] Li LI et al. « Single-Cell RNA-Seq Analysis Maps Development of Human Germline Cells and Gonadal Niche Interactions ». In : *Cell Stem Cell* 20.6 (2017), 858-873.e4. DOI : [10.1016/j.stem.2017.03.007](https://doi.org/10.1016/j.stem.2017.03.007).
- [3] Yijie ZHANG et al. « Single-cell RNA sequencing in cancer research ». In : *Journal of Experimental & Clinical Cancer Research* 40.1 (2021), p. 81. DOI : [10.1186/s13046-021-01874-1](https://doi.org/10.1186/s13046-021-01874-1).
- [4] Atlas M. SARDOO et al. « Decoding brain memory formation by single-cell RNA sequencing ». In : *Briefings in Bioinformatics* 23.6 (2022), bbac412. DOI : [10.1093/bib/bbac412](https://doi.org/10.1093/bib/bbac412).
- [5] Hansruedi MATHYS et al. « Single-cell transcriptomic analysis of Alzheimer's disease ». In : *Nature* 570.7761 (2019), p. 332-337. DOI : [10.1038/s41586-019-1195-2](https://doi.org/10.1038/s41586-019-1195-2).
- [6] Malte D LUECKEN et Fabian J THEIS. « Current best practices in single-cell RNA-seq analysis : a tutorial ». In : *Molecular systems biology* 15.6 (2019), e8746.
- [7] F. Alexander WOLF, Philip ANGERER et Fabian J. THEIS. « Scanpy : large-scale single-cell gene expression data analysis ». In : *Genome Biology* 19.1 (2018), p. 15. DOI : [10.1186/s13059-017-1382-0](https://doi.org/10.1186/s13059-017-1382-0).
- [8] Vladimir Yu KISELEV, Tallulah S ANDREWS et Martin HEMBERG. « Challenges in unsupervised clustering of single-cell RNA-seq data ». In : *Nature Reviews Genetics* 20.5 (2019), p. 273-282. DOI : [10.1038/s41576-018-0088-9](https://doi.org/10.1038/s41576-018-0088-9).
- [9] Romain LOPEZ et al. « Deep generative modeling for single-cell transcriptomics ». In : *Nature Methods* 15.12 (2018), p. 1053-1058. DOI : [10.1038/s41592-018-0229-2](https://doi.org/10.1038/s41592-018-0229-2).
- [10] W Evan JOHNSON, Cheng LI et Ariel RABINOVIC. « Adjusting batch effects in microarray expression data using empirical Bayes methods ». In : *Biostatistics* 8.1 (2007), p. 118-127.
- [11] Ilya KORSUNSKY et al. « Fast, sensitive and accurate integration of single-cell data with Harmony ». In : *Nature methods* 16.12 (2019), p. 1289-1296.
- [12] Brian HIE, Bryan D. BRYSON et Bonnie BERGER. « Efficient integration of heterogeneous single-cell transcriptomes using Scanorama ». In : *Nature Biotechnology* 37.6 (2019), p. 685-691. DOI : [10.1038/s41587-019-0113-3](https://doi.org/10.1038/s41587-019-0113-3).
- [13] Yaroslav GANIN et al. « Domain-Adversarial Training of Neural Networks ». In : *Journal of Machine Learning Research* 17.59 (2016), p. 1-35.
- [14] Songwei GE et al. « Supervised Adversarial Alignment of Single-Cell RNA-seq Data ». In : *Journal of Computational Biology* 28.5 (2021), p. 501-513. DOI : [10.1089/cmb.2020.0439](https://doi.org/10.1089/cmb.2020.0439).

- [15] Qi ZHU et al. « Discriminative Domain Adaption Network for Simultaneously Removing Batch Effects and Annotating Cell Types in Single-Cell RNA-Seq ». In : *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 21.6 (2024), p. 2543-2555. DOI : [10.1109/TCBB.2024.3487574](https://doi.org/10.1109/TCBB.2024.3487574).
- [16] Reut DANINO, Iftach NACHMAN et Roded SHARAN. « Batch correction of single-cell sequencing data via an autoencoder architecture ». In : *Bioinformatics Advances* 4.1 (2024), vbad186. DOI : [10.1093/bioadv/vbad186](https://doi.org/10.1093/bioadv/vbad186).
- [17] James MACQUEEN. « Some methods for classification and analysis of multivariate observations ». In : *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*. T. 1. 14. Oakland, CA, USA. 1967, p. 281-297.
- [18] Vincent D BLONDEL et al. « Fast unfolding of communities in large networks ». In : *Journal of Statistical Mechanics : Theory and Experiment* 2008.10 (2008), P10008.
- [19] Vincent A. TRAAG, Ludo WALTMAN et Nees Jan van ECK. « From Louvain to Leiden : guaranteeing well-connected communities ». In : *Scientific Reports* 9.1 (2019), p. 5233. DOI : [10.1038/s41598-019-41695-z](https://doi.org/10.1038/s41598-019-41695-z).
- [20] Zhaoyu FANG, Ruiqing ZHENG et Min LI. « scMAE : a masked autoencoder for single-cell RNA-seq clustering ». In : *Bioinformatics* 40.1 (2024), btae020. DOI : [10.1093/bioinformatics/btae020](https://doi.org/10.1093/bioinformatics/btae020).
- [21] Xiangjie LI et al. « Deep learning enables accurate clustering with batch effect removal in single-cell RNA-seq analysis ». In : *Nature Communications* 11.1 (2020), p. 2338.
- [22] Tian TIAN et al. « Clustering single-cell RNA-seq data with a model-based deep learning approach ». In : *Nature Machine Intelligence* 1.4 (2019), p. 191-198.
- [23] Hui WAN, Liang CHEN et Minghua DENG. « scNAME : neighborhood contrastive clustering with ancillary mask estimation for scRNA-seq data ». In : *Bioinformatics* 38.6 (2022), p. 1575-1583. DOI : [10.1093/bioinformatics/btac011](https://doi.org/10.1093/bioinformatics/btac011).
- [24] Karl PEARSON. « On lines and planes of closest fit to systems of points in space ». In : *Philosophical Magazine* 2.11 (1901), p. 559-572. DOI : [10.1080/14786440109462720](https://doi.org/10.1080/14786440109462720).
- [25] Ping XU et al. « scCDCG : Efficient Deep Structural Clustering for Single-Cell RNA-Seq via Deep Cut-Informed Graph Embedding ». In : *International Conference on Database Systems for Advanced Applications*. T. 14611. Lecture Notes in Computer Science. Springer, 2024, p. 195-210. DOI : [10.1007/978-981-97-5575-2_11](https://doi.org/10.1007/978-981-97-5575-2_11).
- [26] Lan JIANG et al. « GiniClust : detecting rare cell types from single-cell gene expression data with Gini index ». In : *Genome Biology* 17.1 (2016), p. 144. DOI : [10.1186/s13059-016-1010-4](https://doi.org/10.1186/s13059-016-1010-4).
- [27] Marilisa NERI et al. « CellSIUS provides sensitive and specific detection of rare cell populations from complex single cell RNA-seq data ». In : *Genome Biology* 20.142 (2019), p. 142. DOI : [10.1186/s13059-019-1748-0](https://doi.org/10.1186/s13059-019-1748-0).
- [28] J. LIU et al. « scAIDE : clustering of large-scale single-cell RNA-seq data reveals putative and rare cell types ». In : *NAR Genomics and Bioinformatics* 2.4 (2020), lqaa082. DOI : [10.1093/nargab/lqaa082](https://doi.org/10.1093/nargab/lqaa082).

- [29] Tianyuan LEI et al. « Self-supervised deep clustering of single-cell RNA-seq data to hierarchically detect rare cell populations ». In : *Briefings in Bioinformatics* 24.5 (2023), bbad335. DOI : [10.1093/bib/bbad335](https://doi.org/10.1093/bib/bbad335).
- [30] Yunpei XU et al. « scCAD : Cluster decomposition-based anomaly detection for rare cell identification in single-cell expression data ». In : *Nature Communications* 15.1 (2024), p. 7561. DOI : [10.1038/s41467-024-51891-9](https://doi.org/10.1038/s41467-024-51891-9).
- [31] Ping XU et al. « scCluBench : Comprehensive Benchmarking of Clustering Algorithms for Single-Cell RNA Sequencing ». In : *Proceedings of the AAAI Conference on Artificial Intelligence*. T. 40. 2026, p. 1364-1372.
- [32] Maayan BARON et al. « A single-cell transcriptomic map of the human and mouse pancreas reveals inter- and intra-cell population structure ». In : *Cell Systems* 3.4 (2016), 346-360.e4. DOI : [10.1016/j.cels.2016.08.011](https://doi.org/10.1016/j.cels.2016.08.011).
- [33] Junyuan XIE, Ross GIRSHICK et Ali FARHADI. « Unsupervised deep embedding for clustering analysis ». In : *International Conference on Machine Learning*. PMLR. 2016, p. 478-487.
- [34] Xifeng GUO et al. « Improved deep embedded clustering on images ». In : *Proceedings of the International Joint Conference on Neural Networks*. 2017, p. 2430-2437.
- [35] Mohammad LOTFOLLAHI et al. « Query-to-reference single-cell integration with transfer learning ». In : *Nature Biotechnology* 39.11 (2021), p. 1422-1434.
- [36] Harold W. KUHN. « The Hungarian Method for the Assignment Problem ». In : *Naval Research Logistics Quarterly* 2.1-2 (1955), p. 83-97. DOI : [10.1002/nav.3800020109](https://doi.org/10.1002/nav.3800020109).
- [37] Hua MENG, Chuan QIN et Zhiguo LONG. « scHNTL : single-cell RNA-seq data clustering augmented by high-order neighbors and triplet loss ». In : *Bioinformatics* 41.2 (2025), btaf044. DOI : [10.1093/bioinformatics/btaf044](https://doi.org/10.1093/bioinformatics/btaf044).
- [38] Takuya AKIBA et al. « Optuna : A Next-generation Hyperparameter Optimization Framework ». In : *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019, p. 2623-2631.
- [39] Hoa Thi Nhu TRAN et al. « A benchmark of batch-effect correction methods for single-cell RNA sequencing data ». In : *Genome Biology* 21.1 (2020), p. 12. DOI : [10.1186/s13059-019-1850-9](https://doi.org/10.1186/s13059-019-1850-9).
- [40] Ricardo J. G. B. CAMPELLO, Davoud MOULAVI et Jörg SANDER. « Hierarchical Density Estimates for Data Clustering, Visualization, and Outlier Detection ». In : *ACM Transactions on Knowledge Discovery from Data* 10.1 (2015), p. 1-51. DOI : [10.1145/2733381](https://doi.org/10.1145/2733381).
- [41] Diederik P. KINGMA et Jimmy BA. « Adam : A Method for Stochastic Optimization ». In : *International Conference on Learning Representations (ICLR)*. 2015.
- [42] Isaac VIRSHUP et al. « anndata : Access and store annotated data matrices ». In : *Journal of Open Source Software* 9.101 (2024), p. 4371. DOI : [10.21105/joss.04371](https://doi.org/10.21105/joss.04371).

- [43] Changhu WANG, Zihao CHEN et Ruibin XI. « Feature screening for clustering analysis of count data with an application to single-cell RNA-sequencing ». In : *The Annals of Applied Statistics* 19.4 (2025), p. 2738-2758. DOI : [10.1214/25-A0AS2102](https://doi.org/10.1214/25-A0AS2102).
- [44] Mathilde CARON et al. « Deep Clustering for Unsupervised Learning of Visual Features ». In : *Proceedings of the European Conference on Computer Vision (ECCV)*. 2018, p. 132-149.
- [45] Jing WANG et al. « scSCCNIA : similarity matrix based contrastive clustering with neighbor information aggregation for single-cell RNA sequencing data ». In : *Briefings in Bioinformatics* 27.2 (2026), bbag094. DOI : [10.1093/bib/bbag094](https://doi.org/10.1093/bib/bbag094).
- [46] De-Min LIANG et Pu-Feng DU. « scMUG : deep clustering analysis of single-cell RNA-seq data on multiple gene functional modules ». In : *Briefings in Bioinformatics* 26.2 (2025), bbaf138. DOI : [10.1093/bib/bbaf138](https://doi.org/10.1093/bib/bbaf138).
- [47] Fatemeh S. Hashemi GOLPAYEGANI et al. « Interpretable Self-Supervised Prototype Learning for Single-Cell Transcriptomics ». In : *ICLR Workshop on Learning Meaningful Representations of Life (LMRL)*. 2025.
- [48] Chenxin YI et al. « Benchmarking deep learning methods for biologically conserved single-cell integration ». In : *Genome Biology* 26.1 (2025), p. 398. DOI : [10.1186/s13059-025-03869-z](https://doi.org/10.1186/s13059-025-03869-z).
- [49] Malte D. LUECKEN et al. « Benchmarking atlas-level data integration in single-cell genomics ». In : *Nature Methods* 19.1 (2022), p. 41-50. DOI : [10.1038/s41592-021-01336-8](https://doi.org/10.1038/s41592-021-01336-8).
- [50] Yushan QIU et al. « scTPC : a novel semisupervised deep clustering model for scRNA-seq data ». In : *Bioinformatics* 40.5 (avr. 2024), btae293. DOI : [10.1093/bioinformatics/btae293](https://doi.org/10.1093/bioinformatics/btae293).
- [51] OPENPROBLEMS. *Pancreas Dataset (v1)*. https://openproblems.bio/datasets/openproblems_v1/pancreas. 2026.